



Erasmus Mundus joint
Master in Artificial Intelligence



SAPIENZA
UNIVERSITÀ DI ROMA

Radboud Universiteit



University of Ljubljana



Co-funded by
the European Union

Streamlining Neural Certificate Synthesis

William Bailkoski

Supervisor: Konstantin Kueffner

Co-Supervisor: Prof. Vicenç Gómez

July 2026

Universitat Pompeu Fabra

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Objectives	2
1.3	Structure of the Report	3
2	Preliminaries	5
2.1	Notation	5
2.2	Stochastic Systems	7
2.2.1	Discrete-Time Stochastic Dynamical Systems	8
2.2.2	Transition Kernels and the Markov Chain	8
2.3	Certificates	10
2.3.1	Specification Classes	10
2.3.2	From Lyapunov to Ranking Supermartingales	11
2.3.3	The Drift Condition	13
2.3.4	Verification Problem Formulations	14
2.4	Counterexample-Guided Inductive Synthesis	15
3	Sensitivity and Lipschitz Bounds	18
3.1	The Lipschitz Constant	18
3.2	Estimating the Lipschitz Constant of a General Function	19
3.3	Lipschitz Estimation in Neural Networks	20
3.4	Controlling the Lipschitz Constant	23
3.5	Sensitivity in Certificate Verification	24

3.5.1	The Cost of Verification	24
3.5.2	The Margin Budget	26
3.6	Tightening the Drift-Lipschitz Constant	26
3.6.1	Exact L_D Computation	27
3.6.2	Results	28
3.6.3	Anisotropic constants for free	28
3.7	Discussion	29
4	Benchmarks	31
4.1	Linear Systems	32
4.1.1	LinSwitch	32
4.1.2	LinStoch	33
4.1.3	DoubleWell	34
4.1.4	Pendulum	34
5	Verification	36
5.1	Symbolic Solvers	36
5.1.1	SMT	37
5.1.2	MILP	38
5.2	Abstract Interpretation	40
5.2.1	Interval Propagation	41
5.2.2	Linear Relaxation	41
5.3	Sampling	43
5.3.1	Monte Carlo	44
5.3.2	Multi-Armed Bandit	45
5.4	The Capability Boundary	46
6	Refutation	47
6.1	Optimisers	48
6.1.1	Random Search	48

6.1.2	Grid Search	48
6.1.3	DIRECT	49
6.1.4	AdaLIPO	49
6.1.5	Hill Climbing (HC)	50
6.1.6	Whale Optimisation Algorithm (WOA)	50
6.2	Experiments	51
6.2.1	Hyperparameter Tuning on BBOB	51
6.2.2	Refuting Faux Certificates	53
7	Streamlining CEGIS	56
7.1	Experimental Setup	56
7.2	Results and Discussion	58
7.3	Conclusions	61
7.4	Future Work	61
	List of Figures	63
	List of Tables	66
	Bibliography	68
A	2D Benchmark Phase Portraits	77
B	Examples of Refuter Search Patterns	80

Acknowledgement

I would like to express my sincere gratitude to:

- My supervisor Konstantin Kueffner, for inspiring me to pursue formal methods and encouraging my curiosity as a researcher.
- The EMAI for providing an amazing experience for myself and my peers.
- My friends, family, and loved ones who have supported me throughout the entire process.

Abstract

Formal guarantees for stochastic dynamical systems cannot come from simulation, since no finite set of sampled trajectories can rule out a rare failure. A guarantee must instead be proven, holding over every admissible initial condition and every realisation of the noise. The dominant route is a *certificate*, a function whose existence acts as proof. Learning the certificate as a neural network handles nonlinear and high-dimensional dynamics, with counterexample-guided inductive synthesis (CEGIS) recovering soundness by alternating a learner with a formal verifier. This verifier is the bottleneck: prevailing practice runs a sound and expensive verification on every round, despite most rounds needing only a counterexample.

This thesis separates the two tasks. Refutation, the hunt for a counterexample, may be fast and unsound. Verification, the proof that no counterexample exists, must be sound and is expensive. We survey and implement three sound-verifier families (symbolic solvers, bound propagation, and sound sampling) under a single interface, mapping their capability boundary on a benchmark suite that spans dynamics class, noise model and dimension. Our experiments show the exact symbolic route is inconsistent beyond two noise dimensions, bound propagation alone reaches transcendental dynamics, and no single family covers the whole suite. We then treat refutation as black-box optimisation, benchmarking six optimisers on standard test functions and on *faux certificates* harvested from genuine CEGIS runs. A well-tuned optimiser matches the sound verifier’s counterexample quality at one to three orders of magnitude lower cost. Assembling the two yields the central result. A cheap refuter pre-screen reduces CEGIS to faster rounds followed by a single sound certification call, with speed-ups reaching $15.7\times$ and turning the hardest benchmark from a multi-hour run into a 22-minute one.

Keywords: Certificate Synthesis; Verification; Optimisation

Chapter 1

Introduction

A stochastic system that must not fail cannot be trusted on the strength of a few successful simulations. Simulation samples behaviour; it never rules out the rare trajectory that ends in disaster. Examples of risky systems include a controller keeping a drone aloft, a policy managing financial risk, or a robot navigating on noisy sensors. What such systems demand instead is a *formal guarantee*: a proof that the system meets its required specification with quantified confidence, holding over every admissible initial condition and every realisation of the noise. This thesis is about producing such guarantees automatically, and most importantly about producing them *quickly*.

1.1 Motivation

The dominant route to a formal guarantee is a *certificate*: a single function whose existence witnesses a proof that a system satisfies a property, reducing an intractable question about infinitely many trajectories to a local, checkable inequality on one function [34]. Verifying a candidate then amounts to establishing one inequality everywhere on the domain.

Classical synthesis fixes the certificate’s functional form in advance (for example, a polynomial for sum-of-squares programming [39]) buying soundness at the price of

expressiveness, and scaling poorly beyond low dimensions and simple dynamics. The modern alternative is to *learn* the certificate as a neural network, expressive enough for high-dimensional, non-polynomial, and even learned dynamics, but forfeiting any closed-form guarantee: fitting a network on finitely many sampled states says nothing about the uncountably many states between them. *Counterexample-guided inductive synthesis* (CEGIS) recovers soundness by interleaving a *learner*, which trains the network on a dataset of states, with a *verifier*, which checks the drift condition over the whole domain and, on failure, returns a counterexample for the learner to repair [1, 12]. The loop repeats until the verifier certifies the candidate.

A verifier inside CEGIS is doing two different jobs: while the candidate is invalid, the loop needs only a *counterexample* to train on; only once the candidate is finally valid does it need a *proof* that no counterexample exists. Despite this, the prevailing practice discharges both jobs with a *single* method running every round, doubling as the counterexample generator on the rounds it rejects. Sound verification is expensive because it must reason over the entire domain. Paying that cost every round, when most rounds need only a counterexample, is the bottleneck this thesis sets out to remove.

1.2 Objectives

The guiding hypothesis, stated formally in Chapter 2, is that giving each of these tasks its own dedicated algorithm, a cheap refuter to hunt counterexamples and a sound verifier reserved for the final proof, yields a substantial speed-up. We provide multiple contributions to establish and support this claim.¹

- **A unified survey and implementation of certificate verification for stochastic systems.** We bring three verifier families — symbolic solvers (SMT, MILP), abstract interpretation (the LiRPA family), and sound sampling methods — under a single interface, and map each onto a benchmark

¹An open-source implementation of every verifier, refuter, experiment driver, and figure in this thesis is available at <https://github.com/will-bailkoski/streamlining-neural-certificates>.

suite. The suite spans dynamics classes, noise models, and dimensions, highlighting where each method succeeds and fails.

- **A dedicated treatment of refutation as black-box optimisation.** We cast counterexample search as global optimisation and benchmark six optimisers. Using uninformed, Lipschitz-aware, and population-based search algorithms, we test on a standard benchmark, then on a library of real *faux certificates* harvested from genuine CEGIS runs, finding that a well-tuned refuter matches the verifier’s counterexample quality at one to three orders of magnitude lower cost.
- **Empirical evidence that separating refutation from verification streamlines synthesis.** Placing a fast refuter pre-screen in front of a sound verifier reduces CEGIS to a sequence of cheap search-and-retrain rounds followed by a single certification call. The speed-up scales with the cost of that call, improving the hardest benchmark from a multi-hour run to minutes.
- **Tighter Lipschitz constants for sample-based verification.** Because verification cost grows as a power of the drift-Lipschitz constant, we tighten estimates on the object sample-based verifiers consume: an exact drift-Lipschitz constant for linear systems, computed directly rather than by loose composition, and sound per-axis (anisotropic) constants. The two savings multiply, reducing the number of samples needed to cover a continuous domain by more than an order of magnitude in four dimensions.

1.3 Structure of the Report

Chapter 2 defines the problem: the class of stochastic systems, the ranking supermartingale certificate and its drift condition, the CEGIS loop that synthesises it, and the central hypothesis the rest of the thesis develops. For **Advanced Topics in AI**, Chapter 3 is a focused study of the Lipschitz constant and contributes two methods for tightening the constant for better verification performance. Chapter 4 presents

the benchmark suite, selecting systems from the control and formal-methods literature so that difficulty rises along interpretable axes. The two halves of the loop are then treated in isolation: Chapter 5 surveys and evaluates the sound verifiers, establishing the capability boundary and the cost of a sound call, while Chapter 6 develops refutation as cheap counterexample search and benchmarks the refuters against it. Finally, Chapter 7 assembles the two into the full pipeline and tests the hypothesis directly, measuring the end-to-end speed-up of a refuter pre-screen over a verifier-only baseline.

Chapter 2

Preliminaries

This chapter defines the objects of this thesis. We fix the class of stochastic systems we aim to reason over, then introduce certificates and the specifications they witness, along with defining the scope of this thesis. We then present the machine-learning backbone for synthesising certificates and identify its two distinct goals, refutation and verification. Finally, we state the central hypothesis of the thesis.

2.1 Notation

This section collects the symbols used throughout the thesis; each is defined in full where it first appears. Calligraphic letters denote sets and spaces ($\mathcal{X}$, $\mathcal{W}$, $\mathcal{D}$, $\mathcal{B}$); blackboard letters denote number sets, probability, and expectation ($\mathbb{R}$, $\mathbb{P}$, $\mathbb{E}$); fraktur letters are reserved for the two central structured objects, the Markov chain $\mathfrak{M}$ and the drift $\mathfrak{D}$; italic letters denote scalars, states, and functions. A subscript θ marks a learned object with neural-network parameters θ , and norms $\|\cdot\|$ are Euclidean unless stated otherwise.

<i>Stochastic systems</i>	
$x_t \in \mathcal{X} \subseteq \mathbb{R}^d$	state at time t ; the state space; d the state dimension
$\mathcal{X}_0, \mathcal{X}_G$	the initial set; the goal (equilibrium) set
$w_t \in \mathcal{W}, \mathbb{P}_w, d_w$	noise disturbance; the noise space; its distribution; the noise dimension
$f : \mathcal{X} \times \mathcal{W} \rightarrow \mathcal{X}$	transition function, with Lipschitz constant L_f
$x' \sim P(\cdot \mid x)$	the random successor of x ; the transition kernel
$\mathbf{1}[E]$	the indicator of the event E
$\mathfrak{M}$	the induced Markov chain
$\Omega, \mathcal{F}_t$	the sample space of noise sequences $\mathcal{W}^{\mathbb{N}}$; the natural filtration $\sigma(x_0, \dots, x_t)$
$\varphi, \mathfrak{M} \models \varphi$	a specification; its satisfaction (φ_{reach} reachability, φ_{term} termination)
τ	the stopping time $\inf\{t \geq 0 : x_t \in \mathcal{X}_G\}$
δ	confidence: probabilistic guarantees hold with probability at least $1 - \delta$
<i>Certificates and synthesis</i>	
V, V_θ	a certificate; a neural certificate with parameters θ
ε	the decrease margin of the ranking-supermartingale condition
$\mathfrak{D}(x)$	the (stochastic) drift $\mathbb{E}[V(x')] - V(x) + \varepsilon$; a certificate is valid iff $\mathfrak{D} \leq 0$ outside $\mathcal{X}_G$
$D(x)$	the deterministic drift $V(f(x)) - V(x) + \varepsilon$ (Chapter 3)
$L_V, L_{\mathfrak{D}}, L_D$	Lipschitz constants of the certificate and of the two drifts
$\mathcal{D}$	the learner's training dataset
$[z]_+$	the positive part $\max(0, z)$
$x^\star$	the worst-case state $\arg \max_x \mathfrak{D}(x)$ sought in refutation
<i>Verification</i>	
$\mathcal{B}, B, \text{Diam}(B)$	a grid over $\mathcal{X}$; one of its cells; the cell's diameter
N	the number of grid cells required to certify
b	bins per noise dimension in the MILP noise discretisation (the model carries b^{d_w} cells)
m	Monte-Carlo successor samples per drift estimate

Note that three D's coexist by design and are distinguished typographically: the fraktur $\mathfrak{D}$ is the stochastic drift at the heart of the thesis, the italic D its determin-

istic counterpart used in the illustrations of Chapter 3, and the calligraphic $\mathcal{D}$ the learner’s dataset. Symbols local to a single method keep the notation of their source and are introduced in place. These cover algorithm hyperparameters (the whale optimiser’s a, b, p , AdaLipO’s online estimate $\hat{k}$, and the multi-armed bandit’s top- k), network layer weights W_ℓ , and system-specific constants such as the Lyapunov matrix P of the quadratic certificate $x^\top Px$. This matrix P is distinguished from the transition kernel by the kernel’s explicit conditioning $P(\cdot | x)$.

2.2 Stochastic Systems

Stochastic systems extend classical dynamical systems to model time-evolving processes subject to random perturbation. Real-world systems are rarely fully deterministic: sensor noise, actuator uncertainty, unmodelled dynamics, and environmental disturbances all introduce randomness into observed trajectories. Two broad modelling philosophies exist. In *robust control*, uncertainty is unknown but bounded, and one designs for the worst-case disturbance within some admissible set. In *stochastic control*, uncertainty is drawn from a probability distribution, and one reasons about average-case or probabilistic behaviour. This thesis adopts the stochastic view, which naturally supports quantitative guarantees of the form “the system satisfies property φ with probability at least $1 - \delta$.”

Stochastic systems arise across a wide range of application domains. In robotics and control they model mobile robots navigating under noisy odometry, predictive controllers planning under Gaussian process uncertainty, and safety-critical systems such as spacecraft rendezvous or UAV flight envelope protection, where collision avoidance under uncertainty must be formally guaranteed [36]. In finance, discrete-time stochastic processes model asset price dynamics, and probabilistic constraints such as Value-at-Risk underpin portfolio risk management [27]. Stochastic models also appear in biology (gene regulatory networks, epidemic models) [3] and cyber-physical systems (network-induced packet loss) [43]. A common thread across these domains is the need not just to simulate behaviour, but to certify it. That is, to provide formal guarantees that a system satisfies a safety or performance specifica-

tion with quantifiable confidence [7, 33]. The development of such certificates for stochastic systems is the central concern of this thesis.

2.2.1 Discrete-Time Stochastic Dynamical Systems

The systems of interest are discrete-time stochastic systems, defined as follows:

$$x_{t+1} = f(x_t, w_t), \quad x_0 \in \mathcal{X}_0, \quad (2.1)$$

where $x_t \in \mathcal{X} \subseteq \mathbb{R}^d$ is the state at time t , $f : \mathcal{X} \times \mathcal{W} \rightarrow \mathcal{X}$ is a (possibly nonlinear) transition function, $w_t \in \mathcal{W}$ is a random disturbance drawn i.i.d. from a distribution $\mathbb{P}_w$, and $\mathcal{X}_0 \subseteq \mathcal{X}$ is the set of admissible initial conditions. A *trajectory* is a realisation $\{x_0, x_1, x_2, \dots\} \in \mathcal{X}^\infty$ of this process. Since w_t is random, a trajectory is itself a random object. One may be interested in behaviour over a finite horizon or asymptotically as $t \rightarrow \infty$; the former is more natural for reachability and safety problems of bounded duration, while the latter arises in stability analysis.

When $\mathcal{X}$ is equipped with a norm, one can further speak of regularity properties of the dynamics — notably the Lipschitz continuity that the sample-based methods of Section 5.3 rely on. In particular we assume we have access to the Lipschitz constant of the dynamics L_f (see Chapter 3 for more details).

2.2.2 Transition Kernels and the Markov Chain

The dynamics (2.1) and the noise distribution $\mathbb{P}_w$ together induce a *transition kernel* $P(\cdot \mid x)$: from a state $x \in \mathcal{X}$, the probability that the successor $x' := f(x, w)$ with $w \sim \mathbb{P}_w$ lands in a measurable set $A \subseteq \mathcal{X}$ is

$$P(A \mid x) := \int_{\mathcal{W}} \mathbf{1}[f(x, w) \in A] \mathbb{P}_w(dw), \quad (2.2)$$

the pushforward of the noise distribution through the dynamics. Because the successor depends only on the current state and fresh, independent noise, the process $\{x_t\}_{t \geq 0}$ is a time-homogeneous *Markov chain* on $\mathcal{X}$, which we denote $\mathfrak{M}$ and take as our primary probabilistic object. Every condition in this thesis is expressed through

the *one-step expectation*

$$\mathbb{E}_{x' \sim P(\cdot|x)}[g(x')] = \mathbb{E}_{w \sim \mathbb{P}_w}[g(f(x, w))], \quad (2.3)$$

the expected value of a function g one step into the future from x .

For the martingale arguments of Section 2.3 we occasionally need to condition on the whole past, not merely the present, and doing so properly requires fixing the underlying probability space. For a fixed initial state x_0 , the trajectory is a deterministic function of the noise sequence, so we take as sample space the space of noise sequences $\Omega := \mathcal{W}^{\mathbb{N}}$, equipped with the product σ -algebra $\mathcal{F}$ and the product measure $\mathbb{P}_w^{\otimes \mathbb{N}}$ of the i.i.d. draws. Each state x_t is then a random variable on Ω through the recursion (2.1), and we write $\mathbb{P}(\cdot | x_0)$ and $\mathbb{E}[\cdot | x_0]$ for probability and expectation under this measure.

Definition 2.1 (Filtration, adaptedness, stopping time) *A filtration on a probability space $(\Omega, \mathcal{F}, \mathbb{P})$ is an increasing family $(\mathcal{F}_t)_{t \geq 0}$ of sub- σ -algebras $\mathcal{F}_0 \subseteq \mathcal{F}_1 \subseteq \dots \subseteq \mathcal{F}$. A process $(x_t)_{t \geq 0}$ is adapted to $(\mathcal{F}_t)_{t \geq 0}$ if each x_t is $\mathcal{F}_t$ -measurable, and a random time $\tau : \Omega \rightarrow \mathbb{N} \cup \{\infty\}$ is a stopping time if $\{\tau \leq t\} \in \mathcal{F}_t$ for every t .*

We equip the chain with its *natural filtration* $\mathcal{F}_t := \sigma(x_0, \dots, x_t)$, the smallest σ -algebra making the trajectory up to time t measurable — the information revealed by observing the process to that point. The chain is adapted by construction, and every hitting time $\inf\{t \geq 0 : x_t \in A\}$ of a measurable set $A \subseteq \mathcal{X}$ is a stopping time, since whether it has occurred by time t is determined by $x_0, \dots, x_t$ alone; the stopping time τ of Section 2.3.1 is of exactly this form, which is what entitles us to apply optional stopping to it. Conditional expectation given $\mathcal{F}_t$ is meant in the usual measure-theoretic sense [51]. The Markov property then makes conditioning on the past coincide with conditioning on the present: for any measurable, integrable g ,

$$\mathbb{E}[g(x_{t+1}) | \mathcal{F}_t] = \mathbb{E}[g(x_{t+1}) | x_t] = \mathbb{E}_{x' \sim P(\cdot|x_t)}[g(x')]. \quad (2.4)$$

Conditioning on the past therefore reduces to the one-step expectation from the

current state, so we work with the compact form (2.3) throughout and invoke the filtration only where a martingale theorem explicitly demands it.

2.3 Certificates

A *certificate* is a function whose mere existence witnesses that a system satisfies a property of interest, without ever simulating its trajectories. The idea originates with Lyapunov [34]: to show that a deterministic system is stable it suffices to exhibit a single scalar function $V : \mathcal{X} \rightarrow \mathbb{R}$ that decreases along the dynamics, turning an intractable question about *all* of the infinitely many trajectories into a local, pointwise condition on one function. This reduction — from global behaviour to a checkable inequality — is the entire appeal of the paradigm, and it is what makes certificates amenable to automated synthesis and verification.

This section makes the idea precise for stochastic systems. We first fix the properties we wish to certify (Section 2.3.1); we then develop the certificate that witnesses them — the ranking supermartingale — and reduce its condition to a single checkable *drift* inequality (Sections 2.3.2–2.3.3); finally we frame the verification and synthesis problems this leaves us with (Section 2.3.4), which the remainder of the thesis is devoted to solving.

2.3.1 Specification Classes

Before one can ask whether a certificate exists, one must fix the property to be certified. A *specification* φ is a property of the trajectories of the Markov chain $\mathfrak{M}$; we write $\mathfrak{M} \models \varphi$ when the system satisfies it. Specifications are stated relative to the state space $\mathcal{X} \subseteq \mathbb{R}^d$, an initial set $\mathcal{X}_0 \subseteq \mathcal{X}$ from which the system starts, and a *goal set* (or *equilibrium set*) $\mathcal{X}_G \subseteq \mathcal{X}$ that it should reach; a probabilistic specification additionally carries a confidence level δ . The two specifications of primary interest in this thesis are the following.

Reachability. The *reachability* specification φ_{reach} holds if, starting from any $x_0 \in \mathcal{X}_0$, the trajectory reaches $\mathcal{X}_G$ with high probability:

$$\mathbb{P}(\exists t \geq 0 : x_t \in \mathcal{X}_G \mid x_0) \geq 1 - \delta \quad \forall x_0 \in \mathcal{X}_0, \quad (2.5)$$

for a confidence parameter $\delta \in [0, 1)$. When (2.5) holds we write $\mathfrak{M} \models \varphi_{\text{reach}}$.

Termination. The *termination* specification φ_{term} is stronger: the system reaches $\mathcal{X}_G$ in *finite expected time*. Defining the *stopping time* $\tau := \inf\{t \geq 0 : x_t \in \mathcal{X}_G\}$, it requires

$$\mathbb{E}[\tau \mid x_0] < \infty \quad \forall x_0 \in \mathcal{X}_0. \quad (2.6)$$

Termination implies reachability with probability one (almost-sure reachability), but the converse does not hold in general. In practice, certificates that establish finite expected stopping times are preferable because they carry quantitative information about *how quickly* the system reaches its goal, and it is this specification we target throughout.

Beyond reachability. There exists a wide range of other specifications and certificate types within the formal methods literature. Of particular note are two recurring specifications defined with respect to an *unsafe set* $\mathcal{X}_U \subseteq \mathcal{X}$: *safety* requires the trajectory to avoid $\mathcal{X}_U$ (with high probability), and *reach-avoid* requires it to reach $\mathcal{X}_G$ while avoiding $\mathcal{X}_U$ en route. Both admit certificate characterisations analogous to the supermartingale conditions developed below [49]. The methods we develop extend to safety and reach-avoid without essential modification.

2.3.2 From Lyapunov to Ranking Supermartingales

We now build the certificate that witnesses these specifications, starting from the deterministic case and relaxing it to the stochastic setting.

Lyapunov functions. Consider first a *deterministic* system $x_{t+1} = f(x_t)$ with equilibrium set $\mathcal{X}_G$. A *Lyapunov function* is a continuous $V : \mathcal{X} \rightarrow \mathbb{R}_{\geq 0}$ satisfying

$$V(x) = 0 \quad \forall x \in \mathcal{X}_G, \quad (2.7)$$

$$V(x) > 0 \quad \forall x \in \mathcal{X} \setminus \mathcal{X}_G, \quad (2.8)$$

$$V(f(x)) < V(x) \quad \forall x \in \mathcal{X} \setminus \mathcal{X}_G. \quad (2.9)$$

The decrease (2.9) forces $V(x_t)$ to fall along every trajectory, and since $V \geq 0$ vanishes only on $\mathcal{X}_G$, the system must reach $\mathcal{X}_G$. The certificate has reduced a global question — does every trajectory converge? — to a local, pointwise inequality.

Supermartingales. Under noise the successor is random, so demanding a pointwise decrease at every realisation is too strong: a single noisy step may raise V . The natural relaxation asks V to decrease *in expectation*. Writing x' for the random successor of x , the function V is a *supermartingale* for $\mathfrak{M}$ if

$$\mathbb{E}_{x' \sim P(\cdot|x)}[V(x')] \leq V(x) \quad \forall x \in \mathcal{X} \setminus \mathcal{X}_G. \quad (2.10)$$

By the Markov property (2.4) this is precisely the classical martingale condition $\mathbb{E}[V(x_{t+1}) \mid \mathcal{F}_t] \leq V(x_t)$, so $\{V(x_t)\}$ is a supermartingale in the usual sense and its convergence theory applies: a non-negative supermartingale that vanishes only on $\mathcal{X}_G$ has $V(x_t) \rightarrow 0$ almost surely, and hence reaches the goal with probability one, $\mathfrak{M} \models \varphi_{\text{reach}}$ [51].

Ranking supermartingales. Almost-sure reachability says the system arrives, but not how quickly. Strengthening the relaxation to a *strict* expected decrease by a fixed margin $\varepsilon > 0$,

$$\mathbb{E}_{x' \sim P(\cdot|x)}[V(x')] \leq V(x) - \varepsilon \quad \forall x \in \mathcal{X} \setminus \mathcal{X}_G, \quad (2.11)$$

makes V a *ranking supermartingale*. By Doob's optional stopping theorem [51] it bounds the expected stopping time, $\mathbb{E}[\tau \mid x_0] \leq V(x_0)/\varepsilon < \infty$, so the system

terminates in finite expected time (2.6), $\mathfrak{M} \models \varphi_{\text{term}}$, with an explicit bound. **The ranking supermartingale is the certificate this thesis is concerned with.** It delivers the quantitative termination guarantee of Section 2.3.1, and its defining condition collapses to a single inequality a machine can check.

2.3.3 The Drift Condition

The ranking condition (2.11) still mixes the certificate, the dynamics, and the margin across an inequality. Moving everything to one side turns it into a single scalar function of the state — the *drift* —

$$\mathfrak{D}(x) := \mathbb{E}_{x' \sim P(\cdot|x)}[V(x')] - V(x) + \varepsilon, \quad (2.12)$$

the expected one-step change in V together with the required margin. The ranking supermartingale condition (2.11) is then *exactly* the statement that this drift is non-positive everywhere outside the goal:

$$\mathfrak{D}(x) \leq 0 \quad \forall x \in \mathcal{X} \setminus \mathcal{X}_G. \quad (2.13)$$

Equation (2.13) is the central object of this thesis. A candidate V is a valid certificate precisely when its expected drift is non-positive at every state outside the goal set; **establishing (2.13) for all x — not merely at the finitely many states we can sample — is the single claim that every verifier and refuter in the sequel is built to settle.** Table 1 collects the three conditions and the guarantees they induce.

Certificate	Condition ($x \in \mathcal{X} \setminus \mathcal{X}_G$)	Guarantee
Lyapunov function	$V(f(x)) < V(x)$	convergence to $\mathcal{X}_G$ (deterministic)
Supermartingale	$\mathbb{E}_{x' \sim P(\cdot x)}[V(x')] \leq V(x)$	a.s. reachability, $\mathfrak{M} \models \varphi_{\text{reach}}$
Ranking supermart.	$\mathfrak{D}(x) \leq 0$	finite expected termination, $\mathfrak{M} \models \varphi_{\text{term}}$

Table 1: The certificate conditions and the guarantees they induce; throughout $V \geq 0$ with $V = 0$ on $\mathcal{X}_G$. This thesis targets the ranking supermartingale, whose condition is the drift inequality (2.13).

2.3.4 Verification Problem Formulations

The certificate conditions above leave us with two distinct computational problems. Throughout, a *candidate* is a function $V : \mathcal{X} \rightarrow \mathbb{R}_{\geq 0}$ with $V = 0$ on $\mathcal{X}_G$, and validity means the drift condition (2.13).

Definition 2.2 (The verification problem) *Given a Markov chain $\mathfrak{M}$ on $\mathcal{X}$ with goal set $\mathcal{X}_G$, a margin $\varepsilon > 0$, and a candidate V , decide whether*

$$\mathfrak{D}(x) \leq 0 \quad \forall x \in \mathcal{X} \setminus \mathcal{X}_G,$$

returning VERIFIED if so, and otherwise a counterexample: a state $x^ \in \mathcal{X} \setminus \mathcal{X}_G$ with $\mathfrak{D}(x^*) > 0$.*

For a fixed V this is a *constraint satisfaction* problem over a continuous domain; the challenge is that the condition must be discharged at uncountably many states, not merely those we can sample.

Definition 2.3 (The synthesis problem) *Given $\mathfrak{M}$, $\mathcal{X}_G$ and $\varepsilon > 0$, together with a hypothesis class $\mathcal{V} \subseteq \{V : \mathcal{X} \rightarrow \mathbb{R}_{\geq 0} \mid V = 0 \text{ on } \mathcal{X}_G\}$, find a candidate $V \in \mathcal{V}$ satisfying the drift condition (2.13).*

Synthesis is strictly harder than verification: any solution must in particular be verifiable, and the search ranges over the whole class $\mathcal{V}$ rather than interrogating a single candidate. The choice of $\mathcal{V}$ is the central design decision — this thesis takes $\mathcal{V} = \{V_\theta : \theta \in \Theta\}$, the class of neural networks over a fixed architecture — and the paradigms below differ exactly in that choice.

Approaches to synthesis. Several paradigms have been developed to address these problems.

Sum-of-squares (SOS) programming [39] restricts V to the class of polynomials and encodes the certificate conditions as polynomial non-negativity constraints, which

can be relaxed to a semidefinite programme (SDP) via the Positivstellensatz. SOS methods provide formal soundness guarantees and are well-suited to low-dimensional polynomial systems, but scale poorly with state dimension and cannot represent non-polynomial dynamics.

Template-based synthesis instead fixes a parametric form for V — for instance a linear or piecewise-linear combination of fixed basis functions — and solves for its coefficients, reducing synthesis to a linear or convex feasibility programme when the template and dynamics are compatible. Templates are cheap and interpretable, but only as expressive as the chosen basis, and finding a basis that admits a valid certificate is itself the hard part.

Both paradigms fix the hypothesis class in advance and trade expressiveness for scalability. The alternative this thesis pursues is to *learn* the certificate as a neural network, gaining the expressive power to handle high-dimensional and non-polynomial systems at the cost of the closed-form soundness guarantee — a guarantee recovered through the counterexample-guided loop developed in the next section.

2.4 Counterexample-Guided Inductive Synthesis

The neural route replaces the fixed hypothesis class of the previous section with a *neural certificate* V_θ : a neural network with parameters θ . Its universal approximation capacity makes the class expressive enough for high-dimensional, non-polynomial, and even learned dynamics, but it forfeits the closed-form soundness of SOS or template synthesis — fitting V_θ on a finite set of sampled states says nothing about the uncountably many states between them. *Counterexample-guided inductive synthesis* (CEGIS) [1, 12] recovers a sound guarantee by interleaving learning with formal checking, as illustrated in Figure 1.

The loop. CEGIS seeks parameters θ for which V_θ witnesses the specification, i.e. for which the drift condition (2.13) holds: $\mathfrak{D}(x) \leq 0$ for all $x \in \mathcal{X} \setminus \mathcal{X}_G$. It alternates two components. A *learner* takes a finite dataset $\mathcal{D} \subseteq \mathcal{X}$ of states and trains V_θ to satisfy the drift condition *on* $\mathcal{D}$. A *verifier* then checks whether the

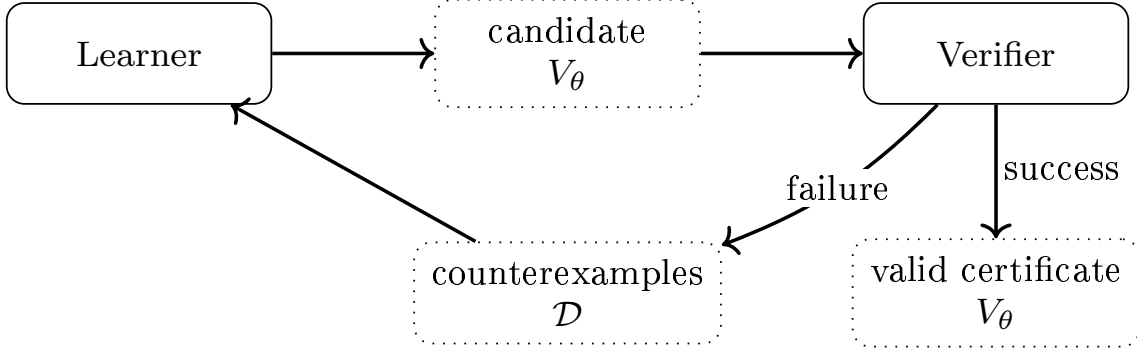


Figure 1: Learner–verifier loop for neural certificate synthesis.

trained V_θ satisfies it on the *entire* continuous domain. If the verifier succeeds, the candidate is a valid certificate and the loop halts. If it fails, it returns one or more *counterexamples* — states at which a condition is violated — which are appended to $\mathcal{D}$ before the learner is invoked again. Each counterexample forbids the current flaw, so the loop makes monotone progress towards soundness.

Learner. The learner performs empirical risk minimisation over the current dataset: it fits the parameters θ so that the candidate satisfies the drift condition at the sampled states. With $[z]_+ := \max(0, z)$ the positive part, it minimises the empirical risk

$$\ell(\theta) = \frac{1}{|\mathcal{D}|} \sum_{x \in \mathcal{D}} [\mathfrak{D}_\theta(x)]_+, \quad (2.14)$$

a hinge penalty that vanishes exactly where the drift $\mathfrak{D}_\theta$ of V_θ (cf. (2.12)) is non-positive. When the one-step expectation inside $\mathfrak{D}_\theta$ has no closed form it is estimated by a Monte-Carlo average over m successor samples $x'_1, \dots, x'_m \sim P(\cdot | x)$. Driving (2.14) to zero certifies nothing on its own: an empirical risk over finitely many states is silent about the continuum between them, which is exactly why a verifier is needed.

Verifier. The verifier discharges the drift condition (2.13) either certifying it, so that V_θ proves $\mathfrak{M} \models \varphi_{\text{term}}$, or returning a counterexample x with $\mathfrak{D}(x) > 0$. Because the claim quantifies over a continuous domain, the verifier must reason soundly about the gaps between sampled states.

Two goals within one loop. The loop of Figure 1 asks its verifier to discharge two formally distinct problems. While the candidate is invalid, the loop needs only to *refute* it. This is an *existential* search for some counterexample (Definition 2.2), which may stop at the first witness and need not be sound, since a spurious near-miss costs only a wasted training point. Once the candidate is valid, the loop needs to *verify* it. This is a *universal* claim that no counterexample exists anywhere, which must clear the whole domain and must be sound, since a single missed violation invalidates the certificate. The prevailing practice nonetheless discharges both with a *single* method, running the sound verifier every round.

Hypothesis. *Because refutation and verification are distinct computational tasks, giving each its own dedicated algorithm yields a substantial speed-up over discharging both with one method.* The remainder of this thesis develops the two tasks separately, verification in Chapter 5 and refutation in Chapter 6, and Chapter 7 tests the hypothesis directly, pairing a fast refuter pre-screen with a sound verifier and measuring the speed-up.

Chapter 3

Sensitivity and Lipschitz Bounds

Advanced Topics in AI contribution

Black-box certificate verification assumes that we only have access to a sample oracle for the dynamics. This poses a unique challenge in ensuring that the certified quantity does not violate our constraints between the finite samples we can actually check. This is closely related to *sensitivity* and *robustness*, and we seek an upper bound on the worst-case change between two point evaluations. In particular, by bounding change with Lipschitz assumptions, we can provide guarantees for neighbourhoods, allowing us to cover a continuous space with single points. This chapter is a focused literature review of what the Lipschitz constant is, how it is estimated and controlled, and why it is the pivotal quantity for the sample-based verification developed later in the thesis. The chapter closes with two small contributions of our own that tighten the constant that sample-based verifiers consume.

3.1 The Lipschitz Constant

A function $f : \mathcal{X} \rightarrow \mathcal{X}$ is *Lipschitz continuous* if some constant L bounds how fast it can change:

$$\|f(x) - f(x')\| \leq L \|x - x'\| \quad \text{for all } x, x' \in \mathcal{X}. \quad (3.1)$$

The smallest such L is *the* Lipschitz constant; any valid L over-estimates it. When f is differentiable, the constant is the largest gradient it attains,

$$L = \sup_{x \in \mathcal{X}} \|\nabla f(x)\| \quad (\text{operator norm of the Jacobian on a convex domain}), \quad (3.2)$$

so estimating L almost always means bounding a supremum of Jacobian norms. For example $f(x) = \sin x$ has $|f'| \leq 1$, so $\sin$ is exactly 1-Lipschitz; $\text{ReLU}(x) = \max(0, x)$ is 1-Lipschitz despite being non-differentiable at the origin, a reminder that Lipschitz continuity does not require differentiability.

Importantly, a small L means a function is not volatile: its output cannot move far when its input is nudged. In machine learning it certifies robustness: a Lipschitz bound and an output prediction margin together lower-bound the perturbation needed to change a network's output [47]. In formal verification, Lipschitz bounds make reasoning over continuous input spaces tractable by extending guarantees from finitely many points to entire regions.

Relevance to certificates. A black-box verifier checks the ranking-supermartingale condition (2.11) only at finitely many sampled states. A Lipschitz bound on the drift function $\mathfrak{D}$ (from (2.12)) allows us to determine the worst-case violation that can occur between samples. This is the principle underlying Lipschitz global optimisation [45]. We therefore seek to calculate $L_{\mathfrak{D}}$ (or find a tight overestimate of it); the looser the bound, the smaller the region that can be certified, making sample-based verification both less informative per sample and more expensive.

3.2 Estimating the Lipschitz Constant of a General Function

Before specialising to networks, we recall what is possible for a general function f , organised by the access we assume. The recurring theme is that *tightness is bought with assumptions*.

Black-box. With only sample access, no sound *upper* bound is possible: finite differences $|f(x_2) - f(x_1)|/\|x_2 - x_1\|$ give a *lower* bound, and between any two samples the true gradient may be arbitrarily large. This is the classical obstruction studied in the complexity theory of real functions [30].

Gradient access. Access to gradients does not rescue the situation: maximising $\|\nabla f\|$ by a first-order method returns another lower bound, because the underlying problem requires global maximisation. Worse, bounding an optimiser’s error is precisely what a Lipschitz constant is for, so estimating L_f this way is circular.

White-box (symbolic access). With the symbolic form of f we are able to produce composition bounds. Given constants L_f, L_g , standard rules give

$$h(x) = af(x) + b, \quad L_h = |a|L_f, \quad (3.3)$$

$$h(x) = f(x) + g(x), \quad L_h \leq L_f + L_g, \quad (3.4)$$

$$h = f \circ g, \quad L_h \leq L_f L_g. \quad (3.5)$$

the last of which underlies the standard neural network bound below. These are upper bounds because they assume the worst-case “stretch” of each factor aligns; that pessimism is the source of most of the looseness this chapter fights.

3.3 Lipschitz Estimation in Neural Networks

Exploiting function structure gives tighter, cheaper estimates. Despite this, in the case of a ReLU feed-forward neural network, the exact constant still remains hard: it is NP-hard even for two-layer networks [48], ultimately because estimating it requires knowing which hidden activation states are reachable, and reachability is NP-hard [32]. The practical methods thus trade tightness against cost; Table 2 summarises what each delivers, and the text expands on the families.

Method	Result	Tightness	Cost
Product of spectral norms	sound upper bd.	loose (trivial)	cheap (per layer)
AutoLip / SeqLip [48]	sound upper bd.	loose–moderate	cheap
LipSDP [16]	sound upper bd.	tight	SDP (scales poorly)
ECLipsE, control-SDP [40, 56]	sound upper bd.	tight	layer-wise / closed-form
IBP [20]	sound local bd.	loose	cheap (intervals)
CROWN / auto_LiRPA [54, 57]	sound local bd.	tighter	moderate
β -CROWN + BaB [50]	complete verdict	exact (in limit)	branch-and-bound
RecurJac, Clarke-BP [44, 58]	sound local bd.	tight	polynomial
Local-drop bound [24]	sound local bd.	tighter than global	cheap
LipBaB [9]	exact , anytime-sound	exact	BaB (worst-case exp.)
Jordan & Dimakis [26]	exact	exact	expensive
Sampling / AdaLIPO [35]	lower bd. only	— (unsound above)	cheap

Table 2: Methods for estimating a network’s Lipschitz constant. *Result* is the kind of estimate returned: a sound upper bound may safely be used for verification, a lower bound may not; “local” methods bound the constant on a bounded region rather than globally (the relevant notion for verification). *Tightness* is how close the estimate comes to the true, smallest constant — looseness is not merely aesthetic, since any over-estimate inflates verification cost through the box count (3.9). *Cost* is the computation needed to produce the estimate, ranging from closed-form per-layer arithmetic to branch-and-bound with worst-case exponential time. LipBaB (bold) is the exact method our experiment builds on.

The product-of-spectral-norms bound. For a linear map $f(x) = Wx$ the Lipschitz constant is exactly the operator norm. Each activation function ϕ contributes its own constant (ReLU and tanh are 1-Lipschitz; the sigmoid is $\frac{1}{4}$ -Lipschitz). Composition rule (3.5) then gives, for an l -layer network,

$$L_f \leq \prod_{\ell=1}^l \|W_\ell\|_2 L_{\phi_\ell}. \quad (3.6)$$

This is scalable but universally loose, because it assumes every layer attains its worst-case stretch on the same input without consideration for inter-layer dynamics. This estimation method serves as the baseline we aim to improve on.

Convex relaxations. The first real improvement comes from convex optimisation. LipSDP [16] exploits that activations are slope-restricted — a quadratic constraint — to pose the estimate as a semidefinite program, capturing inter-neuron interactions the product bound ignores and yielding much tighter bounds. Its weakness is scaling; successors decompose the SDP into per-layer sub-problems or closed forms [56] and extend the paradigm to general architectures [40].

Bound propagation. A second family of methods propagates local output enclosures layer by layer. We can use this to soundly estimate an upper bound on the gradient for a subset of the domain and then select the maximum. Furthermore, one can obtain a local Lipschitz constant over a region by relating the output enclosure to the input radius. Interval bound propagation [20] is the cheapest method but also the loosest, as it discards inter-neuron correlations; CROWN [57] keeps them by bounding each activation with linear functions, and auto_LiRPA [54] generalises this to arbitrary computation graphs. Further discussion of these tools can be found in Section 5.2.

Exact and local constants. At the tight end, LipBaB [9] computes the *exact* local Lipschitz constant of a ReLU network by building a tree that uses branch-and-bound over activation regions, bounding the Jacobian norm on each; it is *anytime-sound*, so early termination still returns a valid upper bound. [26] give a com-

plementary exact method via the generalised Jacobian. Because a global bound over-regularises, several methods instead compute a tighter *local* constant on the reachable region: dropping provably-inactive ReLU units [24], recursive Jacobian bounding [58], or Clarke-Jacobian bound propagation [44]. Such exact enumeration becomes intractable for larger networks, since the number of activation regions grows exponentially with the neuron count.

3.4 Controlling the Lipschitz Constant

Estimation asks how large L is; however, we can skip this challenge by fixing it by design, restricting the hypothesis class to L -Lipschitz networks. The governing tension is expressivity: enforcing per-layer 1-Lipschitzness is easy, but retaining approximation power under that constraint is not [4]. Methods are *soft* (penalties encouraging small L) or *hard* (guaranteeing it by construction).

The softest option adds a *Lipschitz penalty* to the loss function: a regularisation term proportional to a differentiable estimate of the network’s constant — most simply the product of its per-layer spectral norms $\prod_{\ell} \|W_{\ell}\|_2$ — so that minimising the loss also drives L downward. Such penalties are widespread in robust machine learning: the WGAN gradient penalty [21] is a canonical instance, and in the neural-certificate setting a penalty on the network’s Lipschitz bound is routinely added so that the learned certificate (or controller) stays cheap to verify [5]. Being soft, such penalties preserve expressivity but guarantee no bound away from the training distribution.

Hard methods instead build the guarantee into the architecture. The standard choice is spectral normalisation [38], which divides each weight by its spectral norm to make every layer 1-Lipschitz: for $\tilde{W} = W/\|W\|_2$ we have $\|\tilde{W}x\|_2 \leq \|x\|_2$. It is cheap and ubiquitous. Alternatively one can treat the bound as an explicit constraint, projecting the weights back onto the Lipschitz-constrained set after each gradient step [19]. Cruder still is clamping entries to $[-c, c]$, giving $\|W\|_2 \leq \|W\|_F \leq \sqrt{mn}c$, but at the cost of a “uniform shrinking” that guts expressivity. More refined constructions recover expressivity while keeping each layer 1-Lipschitz: orthogonal (norm-

preserving) layers via the Cayley transform [46], the almost-orthogonal rescaling of Prach and Lampert [42], GroupSort networks that are provably universal Lipschitz approximators [4], and the unified SDP view that recovers orthogonal, spectral, and convex-potential layers as special cases and yields the state-of-the-art SLL parameterisation [6]. Coupling control to the guarantee at training time — Lipschitz-margin training — directly enlarges the certified region [47]. Every method is a different answer to the same question: how much expressivity to trade for a guaranteed L .

3.5 Sensitivity in Certificate Verification

We now specialise to the thesis’s setting, where the two disciplines meet: the certificate is an ML object with some sensitivity, and its validity is a formal-methods claim proven from samples. Our goal, to tightly estimate $L_{\mathfrak{D}}$, can now be decomposed into subtasks. By Equation (3.5), we can show that $L_{\mathfrak{D}} \leq L_V \cdot L_f + L_V$. In our problem setting, we assume we have access to L_f , and so the problem reduces to tightly estimating the constant for the certificate, for which we have just established several methods. For the rest of this section we consider deterministic dynamics, which illustrate the ideas more cleanly than the full ranking-supermartingale condition.

3.5.1 The Cost of Verification

Consider a deterministic setting in which we aim to certify termination with a Lyapunov certificate (see (2.9)). To do this, we show that a deterministic drift condition holds on the domain:

$$D(x) = V(f(x)) - V(x) + \varepsilon \leq 0. \quad (3.7)$$

To certify a condition like this, the general procedure is as follows: we discretise the domain into a grid $\mathcal{B}$, and for each cell $B \in \mathcal{B}$ we evaluate its centre point $\text{centre}(B)$. We then determine an upper bound for a grid cell by

$$U(B) = D(\text{centre}(B)) + L_D \times \text{Diam}(B) \quad (3.8)$$

where $\text{Diam}(B)$ is the furthest distance between two points in B . We then certify by checking that $U(B) < 0$; $\forall B \in \mathcal{B}$. This is the basic strategy to certify a continuous condition from finite evaluations; for black-box dynamics this is particularly useful as we only require sample access to D and its Lipschitz constant L_D .

The Lipschitz constant dictates the grid spacing: for a cell to be certifiable, its discretisation error must not swamp the margin, $L_D \times \text{Diam}(B) \leq \varepsilon$. For a hypercube domain this requires

$$N \approx \left(\frac{L_D}{\varepsilon} \right)^d \quad (3.9)$$

cells (Figure 2) to be checked. Because the count is a d -th power of L_D , even a modest tightening of the constant compounds into a large reduction in cost — motivation for both contributions of Section 3.6. (This Lipschitz bridge is one option among several: statistical alternatives instead exploit smoothness via a Gaussian process [10] or derive PAC guarantees from concentration inequalities without any Lipschitz constant [53], and δ -decision procedures resolve a numerical margin [17].)

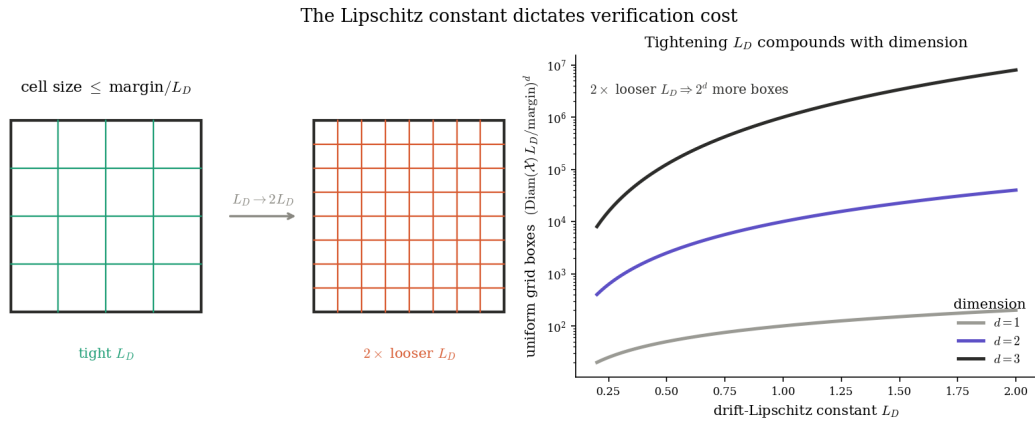


Figure 2: The Lipschitz constant dictates verification cost. *Left:* each cell must be small enough that the drift cannot cross the margin within it, so cell size $\lesssim \text{margin}/L$; a looser constant forces a finer grid. *Right:* the box count (3.9) grows as the d -th power of L , so tightening the constant compounds with dimension.

3.5.2 The Margin Budget

Beyond forcing a finer grid, the Lipschitz constant caps the margin ε one can certify at all. If V is L_V -Lipschitz then $\mathbb{E}_{x'}[|V(x) - V(x')|] \leq L_V \mathbb{E}_{x'}[\|x - x'\|]$, and a valid margin at x obeys $\varepsilon(x) \leq V(x) - \mathbb{E}[V(x')] \leq L_V \mathbb{E}[\|x - x'\|]$; taking the binding (slowest-moving) state,

$$\varepsilon \leq L_V \inf_x \mathbb{E}_{x'}[\|x - x'\|]. \quad (3.10)$$

This illustrates the maximally feasible decrease ε that we can verify for the entire domain. The right-hand side is a property of the *dynamics* (the smallest expected step), so the scale-free invariant is the ratio ε/L_V : a certificate cannot certify more decrease per unit of its own Lipschitz constant than the system actually moves. This highlights the dangers of using Lipschitz-constrained networks as certificates, as an L_V that is too small will violate $\varepsilon/L_V \leq \inf_x \mathbb{E}_{x'}[\|x - x'\|]$, making the verification problem infeasible. Figure 3 illustrates this “budget” empirically; [7] attack the same tension from the certificate side, reparameterising it to lower its theoretical constant.

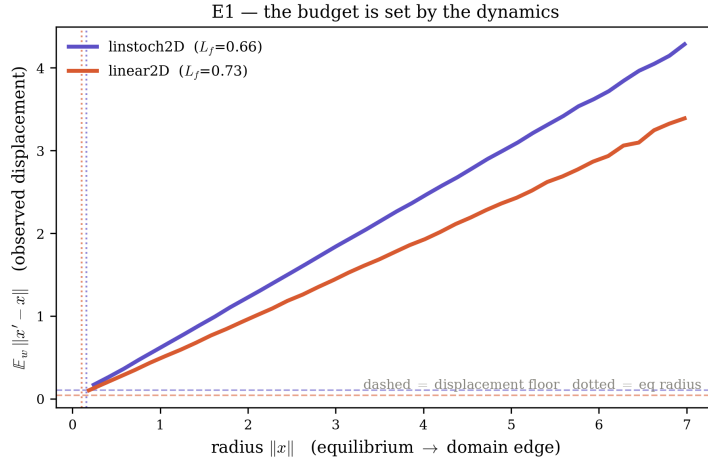


Figure 3: The budget (3.10) is set by the dynamics: the expected one-step displacement $\mathbb{E}_w \|x' - x\|$ rises from a floor near the equilibrium (which defines our domain-wide, fixed ε).

3.6 Tightening the Drift-Lipschitz Constant

The sample-based verifiers in Section 5.3 consume one number: a sound upper bound on the drift-Lipschitz constant $L_{\mathfrak{D}}$. By (3.9) its magnitude drives an estimate

on the cost to certify. To estimate $L_{\mathfrak{D}}$, one might derive a loose product bound $L_{\mathfrak{D}} \leq L_V(L_f + 1)$ via the composition bounds of (3.5) and Section 3.3. However, we have established that composing global bounds is loose, assuming a worst-case alignment that inflates the discretisation error. We develop our tightenings in the deterministic setting of Section 3.5.1, where the drift is D (3.7) and the product bound reads $L_D \leq L_V(L_f + 1)$.

3.6.1 Exact L_D Computation

For this contribution we assume that the dynamics f are deterministic and linear, and V is piecewise linear (for example a feed-forward ReLU network), with constant gradient g_i on each activation region R_i . Given this, ∇D is piecewise constant.

Proposition 3.1 (Exact drift-Lipschitz constant) *For linear A and piecewise-linear V , $L_D = \max\{\|A^\top g_j - g_i\| : R_i \cap A^{-1}R_j \cap \mathcal{X} \neq \emptyset\}$.*

Proof sketch. D is piecewise linear, so its Lipschitz constant is the maximum gradient norm over its linear pieces. Each piece is a nonempty cell $R_i \cap A^{-1}R_j \cap \mathcal{X}$, on which $\nabla D = A^\top g_j - g_i$. $\square$

The exact constant improves on the product bound in two ways: it splits $\|A^\top g_j - g_i\| \leq \|A\| L_V + L_V$ by the triangle inequality (the product bound discards the cancellation between the two terms), and it maximises over i, j independently (the product bound discards that x is the same point in both terms). Soundness of the estimate rests on a simple invariant: for any set S containing every feasible pair, $\max_{(i,j) \in S} \|A^\top g_j - g_i\| \geq L_D$, so starting from all pairs and pruning only *certifiably* infeasible ones can only tighten the bound, never break it. In practice we run LipBaB on the single network $D = V \circ A - V$; being anytime-sound, it returns a valid bound even on early termination.

3.6.2 Results

Table 3 reports three sound upper bounds on L_D for trained certificates on deterministic linear systems in 2–4 dimensions ($\|A\|_\infty < 1$, so the composition bounds are sound over the same box). M1 calculates L_D by computing the product of spectral norms through the network ((3.6)) for L_V , which is then composed with L_f . M2 calculates L_D in the same way, but uses LipBaB to compute the exact constant for L_V . Finally, M3 is the new proposition, where we use LipBaB to compute L_D directly.

Running the branch-and-bound on the drift (M3) is consistently $1.3\text{--}1.5\times$ tighter than the best composition bound (M2) and $2.8\text{--}4.6\times$ tighter than the spectral product (M1). By (3.9) the M2→M3 factor alone divides the box count by $(1.4)^d$, about $2\times$ in 2D and $3.8\times$ in 4D.

d	L_f	M1 spectral	M2 LipBaB	M3 drift	M1/M3	M2/M3
2	0.54	1.10	0.37	0.27	$4.0\times$	$1.4\times$
2	0.60	1.60	0.52	0.38	$4.3\times$	$1.4\times$
3	0.45	1.16	0.55	0.42	$2.8\times$	$1.3\times$
4	0.55	1.78	0.92	0.60	$2.9\times$	$1.5\times$
4	0.51	3.11	0.94	0.67	$4.6\times$	$1.4\times$

Table 3: Sound drift-Lipschitz estimates (upper bounds on L_D). M1 = product of spectral norms; M2 = LipBaB on V composed via (L_f+1) ; M3 = LipBaB run on the drift $V \circ A - V$. M3 is uniformly tightest.

3.6.3 Anisotropic constants for free

Directional constants. Finally, a single global constant can be replaced by *directional* ones. For a unit direction v , $L_f(v) := \sup_x \|J_f(x) v\| \leq L$ is often far smaller, and for a scalar certificate it is one gradient projection to evaluate. We show that LipBaB yields these *soundly*, and that they sharpen the verifier’s discretisation.

LipBaB already computes, per leaf of its search, the exact per-region gradient; the per-axis constant $L_V(e_i) = \sup_x |\partial V / \partial x_i|$ is simply the component-wise maximum of those gradients over the (finitely many, fully enumerated) regions — a *sound* bound satisfying $L_V(e_i) \leq L_V$. These anisotropic constants sharpen the discretisation: over

an axis-aligned cell of side w_i the sound bound $|V(x) - V(y)| \leq \sum_i L_V(e_i) w_i$ replaces the isotropic $L_V \|w\|$, and refining each axis to its own sensitivity ($w_i \propto 1/L_V(e_i)$) reduces the box count from $\propto L_V^d$ to $\propto \prod_i L_V(e_i)$, a factor $\prod_i (L_V/L_V(e_i)) \geq 1$. We report the per-axis constants of the certificate; running LipBaB on the drift network $D = V \circ A - V$ (as in M3) yields the per-axis drift constants $L_D(e_i)$ by the identical extraction, and it is these that enter the box count (3.9).

Table 4 measures this on the same certificates. The per-axis constants are all below the global one and the reduction, being a product over axes, compounds with dimension: from $\approx 1.5\times$ in 2D to $\approx 8\text{--}17\times$ in 4D. Because it multiplies the M2→M3 gain above, the two contributions stack.

d	L_V (global)	per-axis $L_V(e_i)$	box-count reduction
2	0.24	(0.22, 0.18)	$1.5\times$
2	0.32	(0.27, 0.22)	$1.8\times$
3	0.38	(0.36, 0.20, 0.18)	$4.1\times$
4	0.60	(0.32, 0.41, 0.26, 0.23)	$17.0\times$
4	0.62	(0.41, 0.46, 0.40, 0.26)	$7.8\times$

Table 4: Sound anisotropic (per-axis) Lipschitz constants extracted from LipBaB, and the resulting axis-aligned box-count reduction $\prod_i (L_V/L_V(e_i))$. Every $L_V(e_i) \leq L_V$; the saving compounds with dimension.

3.7 Discussion

The two framings of this chapter are one picture. The Lipschitz constant is, to machine learning, a robustness quantity to estimate [9, 16] and control [6, 38]; to formal methods, the bridge that turns finite evidence into a whole-domain guarantee [8, 52]. Certificate verification needs both at once, and because verification cost is a d -th power of the constant (3.9), its magnitude is decisive. There are two levers: shrink the constant by controlling the architecture (Section 3.4), trading expressivity, or estimate it more tightly, changing nothing about the certificate but much about the cost of verifying it. Our two contributions take the second route on the exact object the verifiers consume — the drift constant — computing it directly rather than by composition, and extracting sound per-axis constants that sharpen the grid; because the two savings multiply, they reduce the box count by more than an order of

magnitude in four dimensions. The core limitation is that both lean on the linearity that keeps the drift a ReLU network. Extending them to nonlinear dynamics, where the composed drift is no longer piecewise linear, is the clear next step, and would likely rely on another tool such as bound propagation.

Chapter 4

Benchmarks

In this project we compare the strengths and weaknesses of a range of certificate verification methods. The suite therefore spans a broad range of system types, drawn from the literature to expose where each method succeeds and fails. By identifying structural features of systems that make verification easy or hard, we can separate each method along interpretable axes.

We select systems from the formal-methods and control literature so that the suite spans three orthogonal axes of difficulty:

- **Dynamics class.** From linear, through polynomial, to transcendental (e.g. trigonometric) dynamics. This axis directly controls which symbolic engines can encode the one-step map *exactly*.
- **Noise model.** We assume that the noise distribution is bounded for each system. We include systems ranging from discrete to continuous additive noise variables, and from internal, state-dependent noise to external additive noise.
- **Dimension.** The linear families scale to arbitrary dimension, allowing us to explore how the curse of dimensionality affects each method.

A second selection principle is the **certificate class** each system admits. Some systems have a known, analytic ground-truth certificate (a quadratic $x^\top Px$, an L_1

piecewise-linear function, or a physical potential), which lets us measure a verifier against a *known-good* answer; others have no closed-form certificate and the certificate must be learned, exercising the full CEGIS loop.

Each benchmark instantiates the discrete-time stochastic system (2.1): its dynamics and noise model fix the transition kernel $P(\cdot \mid x)$ of Section 2.2.2, and with it the one-step expectation inside the drift (2.12) that every verifier of Chapter 5 must bound. The selection axes are therefore precisely the levers that control how hard that expectation is to work with: the dynamics class governs whether the one-step map can be encoded symbolically, the noise model governs whether the expectation is an exact finite sum or must be soundly over-approximated, and the dimension scales both.

Table 5: The benchmark suite across the three selection axes.

System	Dynamics	Noise	Analytic Certificate
LinSwitch	PWL / jump	multiplicative (switch)	L_1 PWL (exact)
LinStoch	linear	additive bounded	quadratic $x^\top Px$
DoubleWell	polynomial	additive bounded	potential
Pendulum	transcendental	additive triangular	none

Phase portraits of all four 2D systems (the expected drift field with sample trajectories overlaid) are collected in Appendix A.

4.1 Linear Systems

4.1.1 LinSwitch

The randomly switched linear system is a canonical model in the stochastic stability literature, where at each step the dynamics matrix is selected by an i.i.d. Bernoulli switch:

$$x_{t+1} = A_{\sigma_t} x_t, \quad \sigma_t \sim \text{Bernoulli}(p).$$

We instantiate the randomly switched systems framework of Chatterjee & Pal [11], who give conditions for almost-sure and L_1 stability of such systems. Our 2D instance uses a pair of matrices chosen to satisfy their contractivity conditions, being

jointly contractive in the L_1 norm. Randomly switched systems model phenomena such as mode-switching control, packet loss over lossy channels, and Markov-jump linear dynamics. A phase portrait of the 2D instance is given in Figure 12.

This is the *simplest* stochastic obstruction in the suite and our soundness baseline. The switch is an *internal, discrete* source of randomness, so it is encoded as a small set of exact finite branches rather than an over-approximated continuous noise box, leaving no relaxation anywhere in the pipeline. Every engine handles it, which makes it the control case against which we measure. The L_1 -contractivity gives an *exact piecewise-linear* ground-truth certificate $V(x) = |x_0| + |x_1|$, so the verification conditions hold with *zero relaxation gap* in both SMT and MILP, a property no other system in the suite shares. The construction also generalises. A random-construction variant draws two matrices at a target spectral norm < 1 in any dimension, for which $V(x) = \|x\|_2$ is valid, letting us probe dimensional scaling.

4.1.2 LinStoch

The additive-noise linear system,

$$x_{t+1} = Ax_t + w_t, \quad w_t \sim \text{bounded zero-mean noise},$$

with A Schur-stable, is *the* canonical 2D example of the stochastic-supermartingale literature [7, 33]. The deterministic part is contractive, so the only obstruction to stability is the noise floor. Figure 13 shows the resulting inward spiral to the noise-floor neighbourhood.

It is the additive-noise counterpart to the LinSwitch and isolates the *sound treatment of continuous additive noise*. The noise distribution is over-approximated by a discretisation into cells so that the symbolic engines soundly bound the expected drift. It is also the natural scaling testbed, since A can be constructed at any dimension and target spectral norm.

It admits an *analytic* quadratic ground-truth certificate $V(x) = x^\top Px$ with P the solution of the discrete Lyapunov equation $A^\top PA - P = -Q$ ($Q = I$). The expected

drift then has the closed form

$$\mathbb{E}[V(x')] - V(x) = -x^\top Qx + \text{tr}(P\Sigma_w),$$

which is ≤ 0 exactly outside a ball of radius $r^* = \sqrt{\text{tr}(P\Sigma_w)/\lambda_{\min}(Q)}$. The equilibrium radius is therefore *derived from the noise level*, so V is a genuine supermartingale on domain $\setminus$ equilibrium (the textbook “a.s. stable to a neighbourhood”).

4.1.3 DoubleWell

A 2D stochastic gradient-flow system on a tilted double-well potential,

$$x_{k+1} = x_k - \eta \nabla U(x_k) + w_k, \quad U(x, y) = \frac{1}{4}(x^2 - 1)^2 + \frac{1}{2}y^2 - 0.3x.$$

The double-well is the canonical model of metastability and noise-driven escape over a potential barrier, originating with Kramers [31] and recurring throughout the Lyapunov-certificate literature as a bistable benchmark. Figure 14 shows the flow around the off-origin attractor.

It is the first *nonlinear* (polynomial) system, while still admitting an exact symbolic encoding (the cubic gradient is encoded via an auxiliary squared variable in Z3/Gurobi). It tests whether a verifier handles a non-convex value landscape and a non-origin equilibrium. The potential is *bistable*. The tilt $-0.3x$ breaks the symmetry so one well is the true attractor, but the second (metastable) well is a basin the certificate must rule out. The potential U itself is the ground-truth Lyapunov function, giving an interpretable, physically meaningful V . The equilibrium is off-origin (at $\approx (1, 0)$), unlike every other system.

4.1.4 Pendulum

The inverted pendulum is the archetypal control benchmark. We take the FOSSIL `SineModel` pendulum with its verified LQR feedback gain $K = [[7.21, 1.34], [1.34, 0.33]]$ [1, 15], folded into autonomous closed-loop dynamics, with additive triangular process

noise:

$$\theta' = \theta + h(\dot{\theta} + u_1), \quad \dot{\theta}' = \dot{\theta} + h\left(u_2 + \frac{mgL \sin \theta - b\dot{\theta}}{mL^2}\right), \quad u = -Kx.$$

This is the first *transcendental* system. The $\sin \theta$ term means the SMT and MILP engines cannot encode the map and must abstain, so only autoLiRPA (which relaxes $\sin$) and the sampling/MAB engines apply. It also represents dynamics found in the control literature, in which a controller returns actions to influence a trajectory. Here u is the controller and represents a torque applied to the pendulum, and the equilibrium is its upright position, which would otherwise be unstable if the controller was poor. The closed loop is Schur-stable but *non-normal* ($\|A_d\|_2 > 1$, transient growth), so neither a box nor a ball is forward invariant, and the domain is the Lyapunov sublevel ellipsoid of the linearisation. Nonlinear dynamics and a controller add complexity beyond that of the previous systems. Figure 15 shows the closed-loop spiral over the ellipsoidal domain.

Chapter 5

Verification

Verification asks whether the candidate certificate V satisfies the drift condition everywhere on the domain (Definition 2.2). Each method answers this question in its own way, and the methods differ sharply in what they demand and what they return. We group them into three families. *Symbolic* methods encode the certificate and the dynamics exactly and search for a violating assignment; they need full white-box access and struggle with scaling. *Abstract interpretation* propagates sound over-approximations of the network through each layer; it requires a computation graph for the condition of interest. A spectrum of methods allows tuning between tightness and speed. *Sampling* methods evaluate the drift only at points, so they work for black-box dynamics, at the cost of a probabilistic rather than exact guarantee.

5.1 Symbolic Solvers

With symbolic methods, the main challenge is encoding. We require full, white-box access in order to precisely encode each constraint. This includes full access to the underlying dynamic system, including any noise, and the candidate certificate.

Symbolic methods feature heavily in deterministic verification settings [1, 2, 29, 41]; supermartingales, however, require the encoding of an expected value. For discrete noise distributions this is manageable: one enumerates the outcomes and takes

the weighted average. In many cases, however, the noise is continuous, and no finite formula represents its expectation exactly. In this case, we employ a common strategy of discretising the noise pdf and over-approximating it in order to soundly reason over it [33, 49].

5.1.1 SMT

Satisfiability Modulo Theories (SMT) directly extends Boolean satisfiability (SAT), the problem of finding a truth assignment satisfying a propositional formula. By allowing variables to range over richer domains governed by background theories such as nonlinear real arithmetic, we seek to find a satisfying assignment. In our context, we look for an assignment satisfying the negation of the supermartingale condition; it is this assignment that forms a counterexample.

$$\exists x \in \mathcal{X} : \mathfrak{D}(x) > 0$$

We use Z3 as our SMT backend [13]. The ReLU certificate is piecewise-linear, so it encodes exactly: each neuron becomes a case split, and the expected value in the drift is a weighted sum over the noise branches. Z3 reasons in exact arithmetic [28], so when it returns a verdict there is no numerical slack to worry about.

Z3’s nonlinear real arithmetic is a sound and complete decision procedure for polynomial constraints, so quadratic and even cubic dynamics such as those in DoubleWell are handled exactly. The main difficulty lies in scalability. Even on minimal problems (such as a tiny trained network of two layers of four neurons), Z3 frequently returns *unknown* or exhausts its time budget (3600s).

Compounding this, continuous noise cannot be represented exactly, but it can be soundly over-approximated by binning it into a finite set of branches with a conservative encoding whose slack shrinks as the number of bins grows. This also grows the number of variables that Z3 must check, worsening the scalability wall above.

In addition, Z3 cannot handle transcendental relationships, which restricts both the

dynamics we can certify (Pendulum system requires $\sin(x)$) and the certificates we can check (no sigmoid or tanh network architecture). Tools built on δ -complete decision procedures, such as dReal [18], can reason over these transcendental functions, but only up to a user-specified numerical error rather than exactly.

5.1.2 MILP

Mixed Integer Linear Programming (MILP) extends beyond satisfiability to optimisation problems. By combining propositions and numerical constraints, we represent the verification problem as maximising the drift $\mathfrak{D}$ (see Chapter 6 for more details on defining optimisation problems), and if there exists a solution greater than 0, we reject the candidate.

We use Gurobi (version 13.0.1, via `gurobipy`) as our MILP backend [22]. Casting verification as an optimisation rather than a satisfiability query has a practical benefit: the solver returns the worst-case drift as a number, so we learn not just *whether* the certificate fails but by how much. The ReLU certificate encodes with the usual indicator and max constraints.

As with SMT, polynomial dynamics remain within reach, even though the tool is designed for piecewise-linear problems.

Enabling Gurobi’s nonconvex mode turns it into a spatial branch-and-bound solver: it encloses each product term in a convex relaxation and refines the variable ranges until the relaxation is tight, converging to the global optimum. This is what lets it verify the cubic DoubleWell exactly. One caveat unique to Gurobi is that it works in floating point, so its guarantee holds only up to the solver’s numerical tolerances. Otherwise, the model grows with the network width and the number of noise bins, echoing the scalability issues of SMT. As before, transcendental dynamics remain out of scope. Noise is handled exactly as in the SMT case: discrete outcomes enumerated, continuous noise binned into a sound over-approximation.

The number of bins used to discretise the noise distribution is the crux of the method’s behaviour on stochastic dynamics. Writing b for the bins per noise dimen-

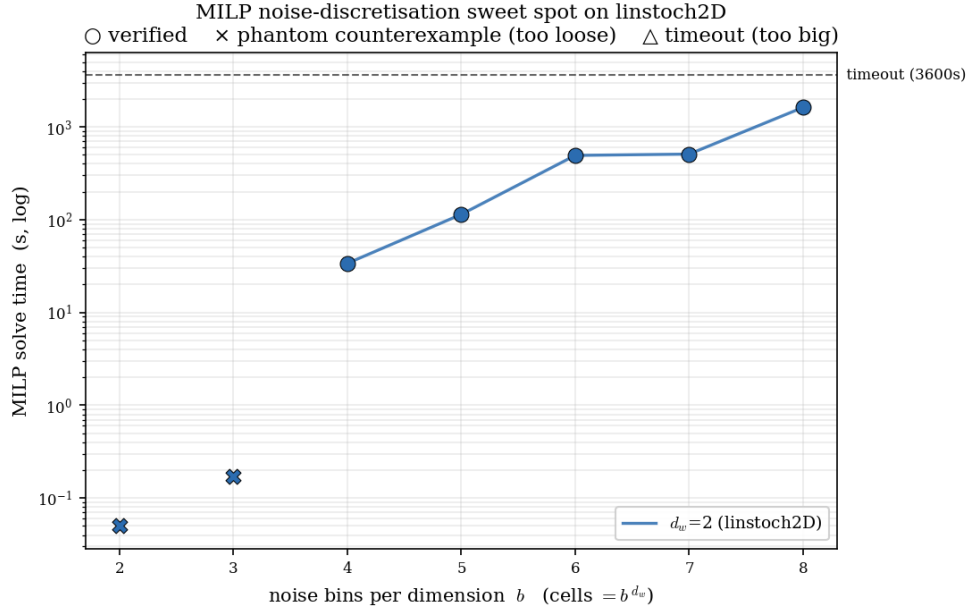


Figure 4: MILP verification of a stochastic-linear certificate on the two-dimensional LinStoch system, against the noise discretisation b (bins per noise dimension, so the model carries b^{d_w} noise cells). The *phantom* regime below $b = 4$ and the steep growth in solve time above it (the cell count itself grows as b^2) are discussed in the text.

sion and d_w for the dimension of the noise variable w , the expectation $\mathbb{E}_{x' \sim P(\cdot|x)}[V(x')]$ is over-approximated by b^{d_w} cells, each carrying its own copy of the successor-network encoding, so the program grows as b^{d_w} (Figure 5). With too few bins the over-approximation is so loose that its drift exceeds the threshold even for a valid certificate: the maximiser returns *phantom* counterexamples — a state whose true expected drift is negative but whose binned over-approximation is positive — and the CEGIS loop is sent chasing a violation that does not exist. With too many bins the b^{d_w} network copies make the program so large that the solver can no longer close the optimality gap within the time budget. Only an intermediate *window* of b both tightens the over-approximation enough to certify and stays small enough to solve.

Figure 4 traces this behaviour on LinStoch in two dimensions. Below $b = 4$ the over-approximation is so loose that every candidate is rejected by a phantom counterexample; from $b = 4$ onward the certificate verifies. The cell count grows as b^2 , and the solve time climbs far faster still, from 34s at $b = 4$ to some 27 minutes at

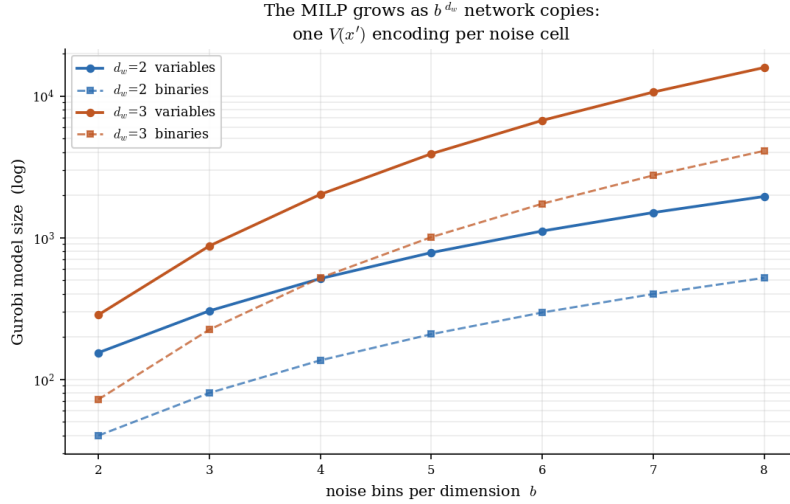


Figure 5: The Gurobi model size (decision variables and binaries) grows as b^{d_w} , since each of the b^{d_w} noise cells must represent and optimise over its own copy of the successor-network encoding. Tightening the over-approximation (larger b) and staying solvable are therefore in direct conflict, and the conflict is exponential in the noise dimension d_w .

$b = 8$. No bin count in the swept range exhausts the one-hour budget, so in two dimensions the binding constraint is the phantom threshold below, not a timeout above.

The window nonetheless closes with dimension. Because the cell count is b^{d_w} , the same sweep in three noise dimensions never escapes the phantom regime: every bin count we could solve, up to $b = 8$ (512 cells), still returns a phantom counterexample, while the b^3 growth drives a single solve past 1600 s by $b = 7$. No bin count is at once tight enough to certify and cheap enough to solve, so the exact route stalls without ever producing a valid certificate. This places the exact route’s limit between two and three noise dimensions, consistent with the frontier the stochastic-certificate literature places at a handful of dimensions [7, 33].

5.2 Abstract Interpretation

Rather than encode the problem exactly, abstract interpretation over-approximates it. Given an input region, we propagate *bounds* through a computation graph, so that the true output is always guaranteed to lie within the computed enclosure.

This makes the method sound by construction: if the enclosure of the drift stays below the threshold, no input in the region can violate it. The price is looseness (the enclosure is wider than the true reachable set), and the central trade-off is how much tightness we are willing to pay for. We use the `auto_LiRPA` framework [54], which spans this spectrum from cheap interval bounds to tighter linear relaxations.

5.2.1 Interval Propagation

Interval bound propagation [20] is the simplest instance: we carry an interval $[\underline{z}, \bar{z}]$ for each neuron and push it through the affine and activation layers. It is cheap and unconditionally sound, but also the loosest option. Because it tracks each neuron independently, it ignores the correlations between them: every ReLU that straddles zero is relaxed in isolation, so the intervals widen quickly with depth and width. On our trained certificates the enclosure is usually too loose to verify without extensive splitting of the input region.

5.2.2 Linear Relaxation

CROWN [57] tightens this by propagating *linear* bounds rather than intervals: each neuron is bounded above and below by an affine function of the inputs, which preserves the inter-neuron correlations (unlike IBP) and yields far tighter enclosures at a moderate cost. Where a single pass is still too loose to decide a region, we close the gap with branch-and-bound, splitting the region and bounding each piece. Optimising the relaxation slopes of each neuron by gradient ascent — α -CROWN [55] — yields a still-tighter bound at a higher per-call cost, and is the strongest of the three engines we use. One practical wrinkle is that the bound-propagation library only supports a fixed catalogue of operations, so a certificate’s output activation must be expressed in terms it recognises: our hard-sigmoid activation function, for instance, is decomposed into a scaling and a clamp. Figure 6 shows this branch-and-bound partition evolving on the Pendulum certificate, each cell shaded by whether its sound enclosure has yet cleared the threshold.

CROWN tools such as `jax_verify` [14] and `auto_LiRPA` require noise to be over-

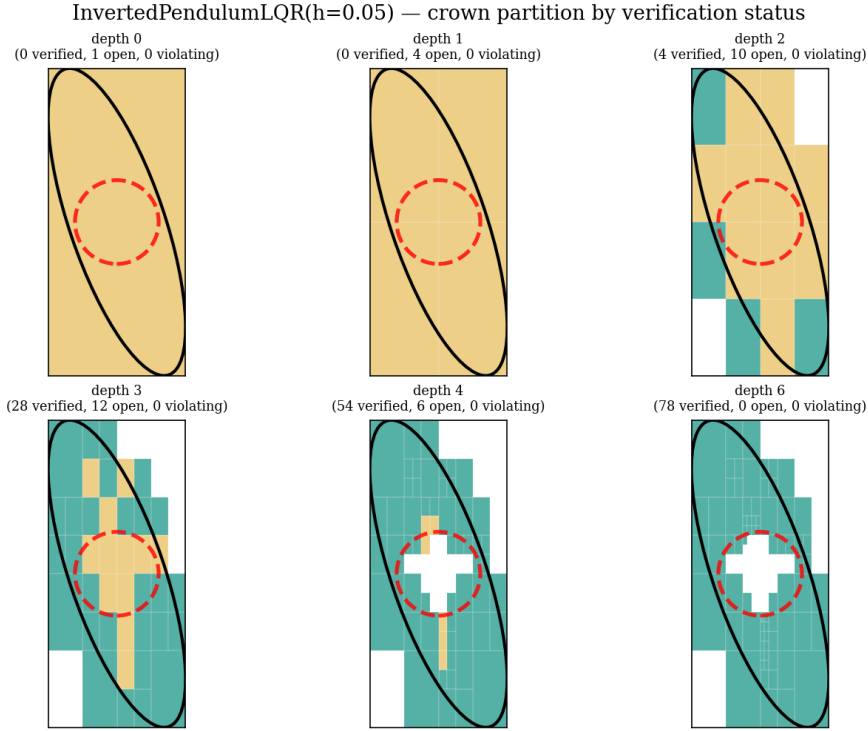


Figure 6: Branch-and-bound in action: CROWN’s partition of the 2D Pendulum state space at successive depths, for a verified certificate. Each cell is shaded by its sound status — *verified* (drift upper bound ≤ 0 , teal), *undecided* (amber, still to be split), or *violating* (red); the black curve marks the domain boundary and the red dashed circle the goal region. From a single undecided box, auto_LiRPA repeatedly halves the cells it cannot yet decide until the whole domain is verified. Tighter relaxations reach this state with far fewer splits than IBP — the compute that the anytime view of Figure 7 prices.

approximated through binning, as with the symbolic methods. Furthermore, they are not natively designed to handle discrete noise, requiring the spawning of multiple separate process threads to evaluate.

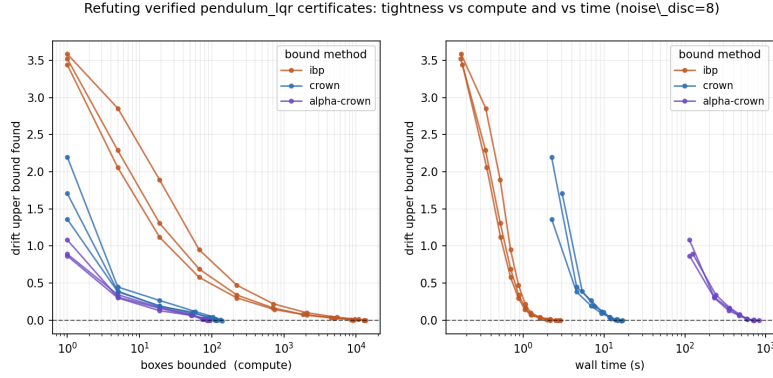


Figure 7: Anytime tightness of the three bound-propagation engines on a verified Pendulum certificate: the sound drift upper bound against wall-clock as branch-and-bound refines. IBP is cheapest per unit compute but loosest, requiring the most domain partitions; α -CROWN reaches the tightest bound at the highest cost; CROWN sits between. All three are sound at every step, as the bound only tightens.

Anytime Tightness. Every engine in this family only ever *tightens* its sound bound as branch-and-bound refines, so it can be halted at any point and still return a valid enclosure — an *anytime* guarantee that the drift $\mathfrak{D}$ exceeds the current upper bound nowhere. In particular, tighter bounding means less branching is required, and so fewer domain regions to check. Figure 7 makes the resulting speed–tightness trade concrete on a verified Pendulum certificate: IBP is the loosest at any compute budget, CROWN tightens it, and α -CROWN reaches the tightest bound at the highest cost. This anytime soundness is precisely the property the CEGIS loop of Chapter 7 exploits, since a verifier call can be stopped the moment its bound crosses zero.

5.3 Sampling

Sampling methods sit at the opposite extreme from symbolic ones [49]. They are the easiest to implement, since they need only sample access to $\mathfrak{D}$: any form of noise or dynamics can be sampled without ever representing its structure, and that flexibility is the whole appeal. The difficulty lies in providing a rigorous guarantee

from finitely many points; an average over a finite set of points says nothing about the gaps between them. What makes these methods sound is a *regularity* bridge, the drift-Lipschitz constant $L_{\mathfrak{D}}$, which bounds how much the drift can change between samples; and even then the guarantee is *probabilistic*, holding with a chosen confidence $1 - \delta$. For deeper discussion on estimating the Lipschitz constant, see Chapter 3. We discuss two methods below, a global Monte Carlo estimate and an adaptive multi-armed-bandit search.

A sampling verifier evaluates $\mathfrak{D}$ only at finitely many points, so the whole difficulty is bridging soundly from those points to the continuum between them. Both methods below do so with a rigorous *drift-Lipschitz* constant $L_{\mathfrak{D}}$; as shown in Chapter 3, a valid choice is $L_{\mathfrak{D}} = L_V(L_f + 1)$, where we assume knowledge of a sound one-step Lipschitz constant of the dynamics L_f , and L_V is a bound on the certificate's Lipschitz constant.

Each method relies on a simple concept: the confidence bound for a grid cell $B \in \mathcal{B}$ is derived from a statistical error as well as a discretisation error. The discretisation error is defined as the product of the diameter of B and $L_{\mathfrak{D}}$, as we have seen before. The statistical error relies on Hoeffding bounds to estimate the confidence that the observed mean of the samples in B reflects the true mean. This requires normalisation with respect to the range of values that $\mathfrak{D}$ can assume, which is non-trivial. To combat this, we restrict the class of V_{θ} that we can learn by constricting the output with either a clamp or a sigmoid activation. In our experiments, we clamp values in $[0, 1]$ to avoid this normalisation step.

5.3.1 Monte Carlo

The Monte Carlo method is heavily inspired by the results and methods discussed in Chapter 3. We first discretise the domain uniformly according to $L_{\mathfrak{D}}$, and then take samples within each grid cell. We track the observed empirical mean of the drift on each cell B and its statistical error to produce upper and lower confidence bounds. If the lower confidence bound for some B is greater than zero, then we certifiably have a counterexample inside B . We return B as a certified violating region and sample

within it to supply the learner’s next counterexamples. If the upper confidence bound is below zero, then we can certify that our certificate condition holds on B with confidence $1 - \delta$. If the process stalls, the discretisation error needs refinement, which motivates our next method.

5.3.2 Multi-Armed Bandit

We consider the Monte Carlo method to be non-adaptive: it samples uniformly in each grid cell, and the cells are uniformly sized. The Multi-Armed Bandit method seeks to improve on this through adaptive sampling and discretisation. We first consider the domain in its entirety and estimate its upper and lower confidence bounds on the mean drift. Similar termination rules apply; however, in the case where the confidence bound straddles zero (making our cell undecided), we consult the error bounds. If the discretisation error is greater than the statistical error, then we split the grid according to some fixed splitting rule. If the reverse is true, then we take more samples. With this, grid refinement is data-driven: grid cells with strong average decrease are verified early and can be pruned as potential violation locations.

Motivated by sample efficiency, in the case where multiple cells are undecided, we can check the top k cells with the highest upper confidence bounds to act upon. With $k = 1$, we take only the most unsure cell, where further sampling will be most beneficial. With larger k , we act upon more and more cells at once, approaching a more uniform gridding. We therefore see a tradeoff between sample-efficiency and time-efficiency through the parameter k .

Experimentation. This method requires both tight estimation of $L_{\mathfrak{D}}$ and tight bounds on the statistical error. For our experiments, we used LipBaB (M2 in Section 3.6) and an empirical-Bernstein radius for the error bounding. Unfortunately, with loose bounds, we were unable to achieve termination within our sample budget (10^9 samples) for either method consistently on any benchmark.

5.4 The Capability Boundary

Across the suite the three families separate cleanly, and the boundary follows the dynamics class and the noise dimension. The *symbolic* engines are the most precise where they apply but the least available: Z3, though a complete decision procedure for the polynomial systems, returned *unknown* or exhausted its budget on essentially every trained-network round, and Gurobi’s exact route closes between two and three noise dimensions as the discretisation window shuts (Figure 4); neither can encode the Pendulum’s transcendental dynamics, and both abstain there. The *sampling* verifiers are the most general in principle, indifferent to the form of the noise and the dynamics, but sound worst-case sampling proved too expensive in practice: within our budgets neither Monte Carlo nor the multi-armed bandit could be driven to termination, so we do not carry them forward as benchmarked engines. The *bound-propagation* family has the widest coverage and is the only one to reach the transcendental Pendulum — yet it is not universal either: LinStoch, whose noise-floor equilibrium leaves the relaxation little margin to work with, was certified only by the exact MILP route inside its discretisation window (Figure 4). No single family clears the whole suite; the boundary is two-sided, with the exact route alone owning the additive-noise linear system and bound propagation alone reaching past the symbolic frontier. Within the LiRPA family, IBP, CROWN and α -CROWN trade per-call tightness against cost along the anytime curve of Figure 7.

This boundary fixes the cast for the remainder of the thesis. The three LiRPA engines are the sound verifiers of the end-to-end synthesis campaign of Chapter 7, run on the three systems they certify, where their per-call costs (from sub-second for IBP on LinSwitch to some twenty minutes for α -CROWN on the Pendulum) are measured inside the loop. And every round in which a verifier (MILP or LiRPA) rejected a candidate leaves behind a genuine refutation problem; these rejected candidates form the faux-certificate library on which Chapter 6 benchmarks the refuters. What no engine changes is the fundamental expense of a sound call, which must reason over the entire domain; removing that cost from every round of synthesis is the subject of the final chapter.

Chapter 6

Refutation

Refutation is the search for a counterexample that invalidates a candidate certificate. The counterexample is an explicit state in the sense of Definition 2.2 — one violating the condition of interest — which we then use to guide the next round of CEGIS. Counterexamples are much easier to find than proofs of their absence, primarily because the claim changes shape: instead of establishing the condition for *all* $x \in \mathcal{X}$, it suffices to exhibit *some* $x \in \mathcal{X}$ that violates it. This lets us treat the problem as a search, which careful formulation can pose in several efficient ways. In this chapter we benchmark six black-box optimisers, testing them on standard test problems as well as real neural certificates to assess their usefulness as a dedicated subroutine in the CEGIS pipeline.

Let

$$x^\star = \arg \max_{x \in \mathcal{X}} \mathfrak{D}(x).$$

From an optimisation perspective, verification checks

$$\mathfrak{D}(x^\star) \leq 0,$$

whereas any $x \in \mathcal{X}$ such that

$$\mathfrak{D}(x) > 0$$

is in fact a counterexample. Whilst all the aforementioned verification methods can find these counterexamples and serve as refuters, we can also consider specialised refuters that are designed around this optimisation problem.

6.1 Optimisers

Optimisers are an attractive choice for refutation for two reasons. First, global optimisation is a mature field, so there is a large body of prior research to draw well-performing optimisers from. Second, optimisers are readily customised: the objective function, constraints, and search targets can each encode whatever information our assumptions permit, steering the search accordingly.

As this is for refutation and not for verification, strict soundness is no longer of concern, and even points that seem close to counterexamples provide insight into the margin we have certified for. This makes a Monte-Carlo approximation of the expectation good enough for our use case. For each point evaluation, we take m Monte-Carlo samples of the successor state, resulting in the objective function $\tilde{\mathfrak{D}}(x) = \frac{1}{m} \sum_{i=1}^m V(f(x, w_i)) - V(x) + \varepsilon$. Occasionally this can cause spurious counterexamples, leading to bad early-termination or incorrect local maxima.

6.1.1 Random Search

Random search is the most naive method of search in which we take uniform, iid samples across $\mathcal{X}$, tracking only the best evaluations and disregarding the rest. It is simple to implement, extremely fast, but completely uninformed. It effectively acts as a benchmark for counterexample quality: the larger the drift value, the larger the violation, and the better the quality of the counterexample.

6.1.2 Grid Search

Grid search is analogous to a non-adaptive MAB algorithm. We grid the domain and evaluate centres, then refine and repeat uniformly across the domain. As a result, it is often centred on the domain and takes symmetrical evaluations which can

sometimes be beneficial and other times wasteful. With each iteration the number of points to evaluate grows exponentially, so its per-iteration cost climbs with the point count. Both grid search and random search are everywhere-dense in the limit of infinite resources.

6.1.3 DIRECT

DIRECT (DIviding RECTangles) [25] reframes global optimisation as a recursive partition of the search domain into hyper-rectangles, each represented by its centre. At each iteration it identifies the potentially optimal rectangles, balancing exploitation of regions with high centre values against exploration of large, under-sampled regions. Unlike traditional Lipschitz optimisation methods, DIRECT does not require a user-specified Lipschitz constant; instead, it implicitly considers all possible positive Lipschitz constants when selecting rectangles for refinement. The selected rectangles are trisected along their longest dimension(s), and the centres of the resulting subrectangles are evaluated. Because the selection rule simultaneously considers rectangles across multiple scales, DIRECT naturally balances local refinement with global exploration and rarely commits prematurely to a single basin. Aside from a small parameter ε_{po} used to avoid excessive refinement of the current best region, the method is effectively parameter-free. Its main computational cost lies in maintaining the growing set of rectangles and identifying the potentially optimal subset via a convex-hull computation.

6.1.4 AdaLIPO

AdaLIPO (Adaptive Lipschitz Optimisation) [35] is a purely sample-driven Lipschitz method. It keeps a history of evaluated points, and each iteration proposes a large uniform pool of candidates, scoring each by the tightest upper bound its history allows, $\min_i [y_i + \hat{k} \|x - x_i\|]$, and spends its evaluation budget on the most promising ones. The Lipschitz constant $\hat{k}$ is estimated online from the steepest slope seen so far and inflated slightly to stay optimistic. This makes AdaLIPO a natural fit for refutation: it exploits precisely the regularity we already assume of the drift —

its Lipschitz continuity — to steer samples toward the region most likely to hide a violation, rather than searching blindly. The overhead is the pairwise distance computation against the history, which grows with the number of evaluations.

6.1.5 Hill Climbing (HC)

HC exploits differentiability to traverse the landscape, wherein an agent explores in the direction that the derivative is most positive. It is the counterpart to Gradient Descent, which is famously used in machine learning for minimising loss functions. In its most basic form, it is susceptible to getting stuck at local maxima, and as such we employ random restarts to encourage the agent to explore more of the state space. Because JAX provides automatic differentiation and JIT compilation, we run HC efficiently on GPU and vectorise it so that many restarts search at once. Crucially, this gradient access is an extra assumption made only by HC.

6.1.6 Whale Optimisation Algorithm (WOA)

WOA [37] is a population-based metaheuristic inspired by the bubble-net hunting of humpback whales. A population of candidate points moves each iteration by one of three behaviours relative to the incumbent best: a shrinking *encircle*, a logarithmic *spiral*, or an *exploratory* step toward a random peer. A single parameter a anneals linearly over the budget, shifting the population from exploration to exploitation. The spiral gives WOA a distinctive ability to probe around a promising point at many radii at once, and being population-based it is naturally parallel and resistant to being trapped by a single local optimum. It also carries the most tuning knobs of the refuters we consider: the annealing ceiling $a_{\max}$, the spiral tightness b , and the spiral/encircle mix p . Crucially, WOA has a track record as a fast and flexible black-box optimiser, performing consistently in optimiser comparisons on the Black-Box Optimisation Benchmarking (BBOB) suite of the COCO platform [23].

6.2 Experiments

We study the refuters in two stages. Because they cannot certify on their own, their value in the pipeline is as a cheap pre-screen: run up to a fixed budget each round, falling back on the sound verifier only when no counterexample is found — the arrangement Chapter 7 evaluates end-to-end. We first tune each refuter as a black-box optimiser on a standard benchmark suite (Section 6.2.1), then turn the tuned refuters on genuine refutation tasks (Section 6.2.2).

6.2.1 Hyperparameter Tuning on BBOB

Before comparing refuters we must give each a fair chance, which means choosing its hyperparameters well. We take five functions from the BBOB test suite, spanning the landscape archetypes that separate search strategies: Rastrigin (f3, a regular grid of local optima), Rosenbrock (f8, a narrow curved valley), Schaffers F7 (f17, irregular ripples at every radius), Schwefel (f20, best basins are far from the centre), and Gallagher’s 101-peak function (f21, a narrow needle among distractors). Each is negated so that the refuter *maximises* and the global optimum is 0; every function is first checked for well-formedness (Figure 8).

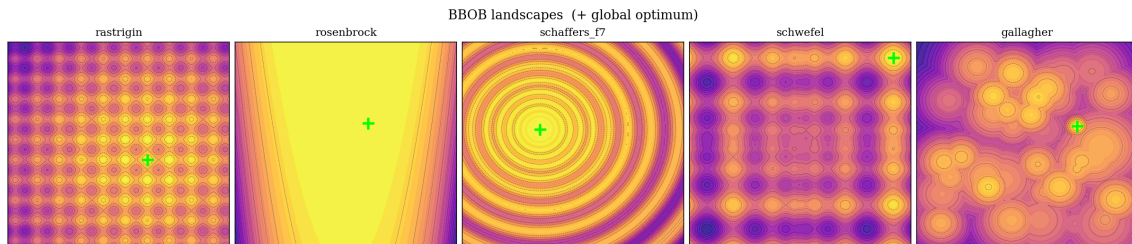


Figure 8: The five BBOB test functions with the global optimum marked in green. Yellow indicates high-quality solutions and purple indicates low-quality solutions.

For each refuter we sweep its hyperparameter grid over the five functions and three seeds at a budget of 8192 evaluations, and score a run by its *normalised regret* $r = -\hat{y}^*/\text{Diam}(f) \in [0, 1]$, where $\hat{y}^*$ is the best value found and $\text{Diam}(f)$ the span of f over the domain, so that $r = 0$ means the optimum was located. The best hyperparameters for a refuter are those minimising the mean regret over functions

and seeds; Table 6 reports them.

Refuter	Best hyperparameters	Mean regret	Time (s)
Whale (WOA)	$a_{\max} = 1.5, b = 0.5, p = 0.75$	5.8×10^{-8}	0.004
DIRECT	$\varepsilon_{\text{po}} = 10^{-3}$	3.7×10^{-5}	0.148
AdaLIPO	pool = 16, $\alpha = 0.2$	4.9×10^{-4}	0.686
Grid	—	2.3×10^{-3}	0.001
Random	—	5.1×10^{-3}	0.002
Hill climbing	lr = 0.5, decay = 0.95	5.7×10^{-3}	0.002

Table 6: Best hyperparameters per refuter, by mean normalised regret over the five BBOB functions and three seeds. Lower regret is better; time is the mean wall-clock per search.

Two results stand out. First, the Lipschitz-aware and population methods dominate the uninformed ones: whale, DIRECT and AdaLIPO reach regrets one to five orders of magnitude below random, grid and hill climbing. Second, accuracy need not be bought with time: whale in particular solves all five landscapes at essentially zero regret in milliseconds, a combination that Figure 10 places on the quality–cost plane below.

Figure 9 breaks the tuned refuters down by function. Rosenbrock’s valley is easy for everyone at this budget; it is the multimodal and deceptive functions that separate the methods. Hill climbing is competitive only on the smooth valley and the ripples, and collapses on Schwefel (two orders of magnitude worse than plain grid search) and Gallagher which punish pure exploitation. Whale and DIRECT solve every function; AdaLIPO is a consistent middle tier. Figure 16 shows how the refuters differ in where they spend their evaluations on a shared landscape, from the uniform scatter of random search to the basin-focused trajectories of the informed methods.

Finally, Figure 10 places quality against wall-clock cost. Two clusters emerge: a fast-and-accurate frontier occupied by whale (millisecond searches at essentially zero regret) and a slower, deliberate cluster (AdaLIPO and DIRECT at tenths of a second, paying their per-iteration Lipschitz bookkeeping for accuracy). Random, grid and hill climbing are fast but sit three to five orders of magnitude worse on the hard functions. Whale at its tuned configuration dominates the frontier outright, and it is these tuned configurations that we carry into the refutation experiment

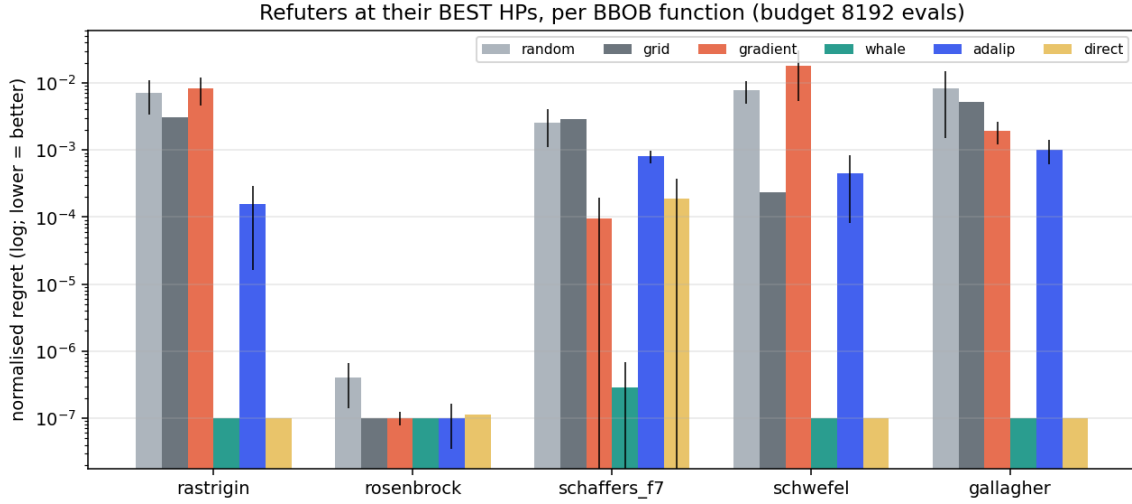


Figure 9: Normalised regret (log scale) of each refuter at its best hyperparameters, per BBOB function. The multimodal and deceptive functions (Schwefel, Gallagher, Rastrigin) separate the informed methods from the uninformed ones.

that follows.

6.2.2 Refuting Faux Certificates

The BBOB benchmark tunes and characterises the refuters as generic optimisers; we now experiment with refuters within the context of our problem definition. Every round in which a sound verifier (MILP or LiRPA) rejected a candidate certificate during the verification runs of the previous chapter left behind a *faux certificate*: a trained network that looks like a valid supermartingale but violates the drift condition somewhere, stored together with the counterexample the verifier found and the wall-clock it took to find it. These harvested faux certificates form a library of genuine refutation problems, drawn from the real CEGIS trajectory rather than a synthetic benchmark.

We replay each harvested faux certificate with all six refuters at their BBOB-tuned hyperparameters (Section 6.2.1), and record, against the sound verifier that originally rejected the candidate, whether each refuter finds a counterexample, how fast, and the *quality* of the counterexample it returns (the drift value it attains under a common high-sample estimator). Our library holds 91 distinct faux certificates

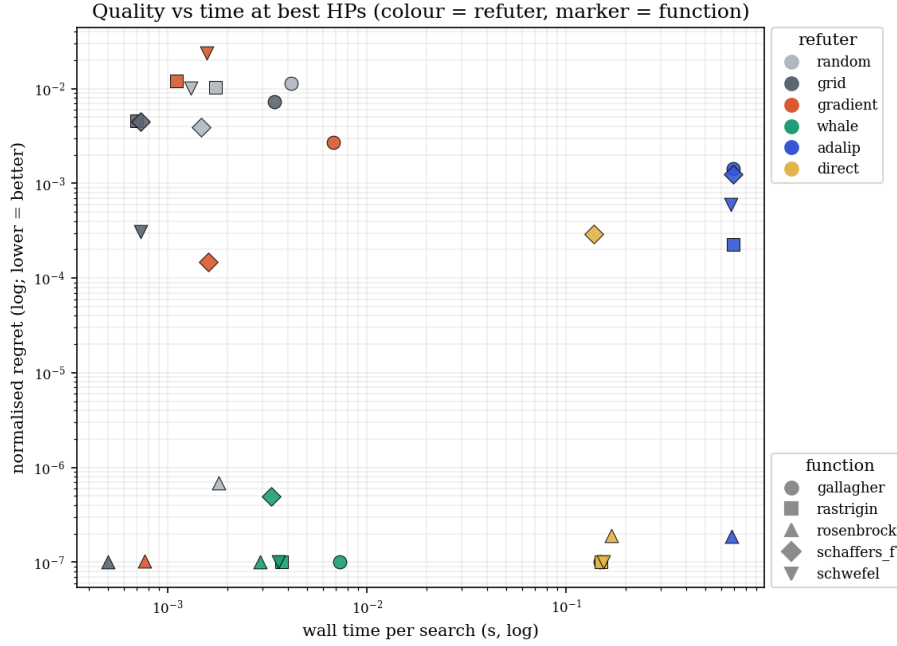


Figure 10: Quality versus wall-clock cost at the best hyperparameters (colour = refuter, marker = function; both axes logarithmic). Whale occupies the fast-and-accurate frontier; the Lipschitz methods trade time for accuracy; uninformed search is fast but far worse on the hard landscapes.

harvested from the LinSwitch, DoubleWell and Pendulum verification runs, each replayed with all six refuters, and the stochastic refuters over several random seeds.

Every refuter recovers the large majority of counterexamples far more cheaply than the sound verifier (Figure 11, Table 7). A single refutation costs between 1 and 21 ms against the verifier’s median 0.48 s, for median speed-ups of 69–352 $\times$. Two axes separate the methods. The first is *reliability*: whale finds a counterexample on 99% of the faux certificates across all three systems and returns the deepest ones (median drift 0.049) while random, grid, AdaLIPO and hill climbing sit at 91–95%. DIRECT is the outlier: strong on the synthetic BBOB suite, it collapses to 40% here, missing two-thirds of the linear-system certificates. The second is *cost*: the uninformed random and grid searches are the fastest per cert (1.4 ms, $\sim 350\times$) because they model nothing, whereas whale’s thorough spiral search costs more (3.9 ms, $\sim 130\times$). This is the empirical basis for the pre-screen: on the certificates a sound verifier would reject, a cheap refuter reaches the same verdict at a fraction of the cost, and a well-chosen one does so almost every time.

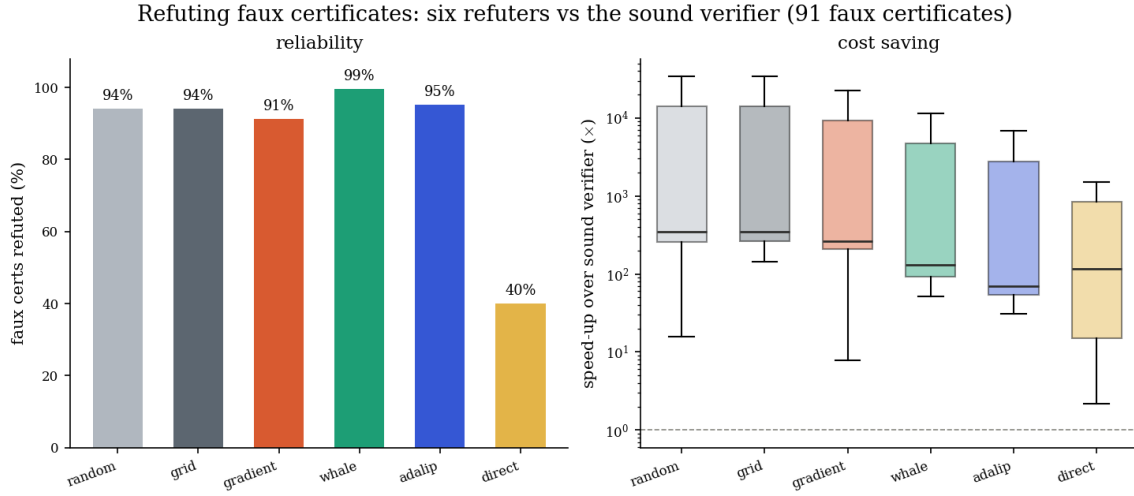


Figure 11: Refuting the faux certificates a sound verifier rejected, six refuters compared. *Left*: the fraction of faux certificates each refuter refutes — whale is near-perfect, the other informed and uninformed methods cluster at 91–95%, and DIRECT collapses to 40%. *Right*: per-refuter speed-up over the sound verifier (log): every refuter is one-to-three orders of magnitude cheaper than a sound call, the fast uninformed searches highest.

Refuter	Success rate	Verifier (s)	Refuter (s)	Speed-up	Quality
Whale	99%	0.50	0.004	131×	0.049
AdaLIPO	95%	0.48	0.007	69×	0.042
Random	94%	0.48	0.001	344×	0.039
Grid	94%	0.48	0.001	352×	0.038
Hill climbing	91%	0.48	0.002	262×	0.038
DIRECT	40%	0.48	0.021	116×	0.024

Table 7: Faux-certificate refutation, six BBOB-tuned refuters versus the sound verifier that rejected each candidate. Columns: the fraction of faux certificates on which a genuine counterexample was found; *median* wall-clock to a counterexample for the verifier and the refuter, and their ratio; and median counterexample quality (scored drift). Whale is the most reliable and returns the deepest counterexamples; the uninformed searches are the cheapest per call; DIRECT misses most of the linear-system certificates.

Chapter 7

Streamlining CEGIS

The preceding chapters characterised the two halves of certificate synthesis in isolation: Chapter 5 the sound verifiers, and Chapter 6 the refuters as cheap counterexample search. This chapter assembles them into the full CEGIS loop and asks: does placing a refuter in front of the sound verifier make end-to-end certificate synthesis faster?

7.1 Experimental Setup

We run the complete CEGIS pipeline on three benchmark systems chosen to span the difficulty spectrum of Chapter 4: the 2D LinSwitch (Section 4.1), the DoubleWell gradient flow (Section 4.1.3), and the LQR-controlled Pendulum (Section 4.1.4), whose transcendental dynamics make bound propagation expensive. The fourth benchmark, LinStoch, is absent by necessity rather than choice: Chapter 5 found it certifiable only by the exact MILP route, and the LiRPA engines this campaign compares were unable to verify it, so no verifier-only baseline exists to improve upon.

The learner trains the ReLU certificate architecture used throughout (two hidden layers of eight units) to a training margin of 0.02, and the loop terminates when a sound verifier certifies the drift condition to $\varepsilon = 10^{-3}$. Any counterexample found, whether by refuter or verifier, is added to the training set and the candidate is

retrained.

For our verifiers we take the three LiRPA-family engines of Chapter 5, the only family that certifies all three systems. All three run inside the same branch-and-bound refinement with longest-edge region splitting. Against each engine we compare three loop configurations, which differ only in how counterexamples are sought:

Verifier-only. The baseline: the sound verifier runs every round, both finding counterexamples and, eventually, certifying.

Random pre-screen. Each round, random search (Section 6.2.1) hunts for a counterexample first; only when it finds nothing do we fall back on the sound verifier. Random is the uninformed extreme: essentially free, but blind.

Whale pre-screen. The same arrangement with the whale optimiser (WOA) at its BBOB-tuned hyperparameters (Table 6), selected because it dominated the fast-and-accurate frontier of the refuter benchmark.

Refuters run with the same budget as in Chapter 6 (8192 evaluations per search, 16 Monte-Carlo samples per point). Each configuration is repeated over five seeds. We record the wall-clock time to a verified certificate, the number of CEGIS rounds, the number of sound-verifier calls, and the number of *escapes* — counterexamples the refuter missed that the sound verifier subsequently found.

Compute environment. Campaigns throughout the thesis were dispatched as job arrays to a Slurm compute cluster; the sampling verifiers (MC, MAB) and the three LiRPA engines benefit from GPU acceleration and ran on GPU nodes. The MILP experiments are the exception, running locally on a desktop CPU, as the Gurobi licence (version 13.0.1, via `gurobipy`) could not be reached from the cluster’s compute nodes. All code (the unified verifier framework, the refuters, the experiment drivers, and every figure script) is available at <https://github.com/will-bailkoski/streamlining-neural-certificates>.

7.2 Results and Discussion

Engine	System	Strategy	n	Time (s)	Rounds	Calls	Speed-up
IBP	LinSwitch	verifier-only	5	155.5	43.8	43.8	—
		random	5	58.9	18.0	2.6	2.64×
		whale	5	39.8	10.8	1.0	3.91×
	DoubleWell	verifier-only	3	14.6	1.3	1.3	—
		random	3	14.1	1.7	1.0	1.03×
		whale	5	18.4	2.2	1.0	1.04×
	Pendulum	verifier-only	1	55.0	6.0	6.0	—
		random	2	33.7	7.5	1.0	1.76×
		whale	1	57.4	10.0	1.0	—
CROWN	LinSwitch	verifier-only	5	44.6	11.8	11.8	—
		random	5	52.2	15.4	2.0	0.85×
		whale	5	38.5	10.8	1.0	1.16×
	DoubleWell	verifier-only	3	41.7	1.7	1.7	—
		random	5	31.3	2.2	1.0	1.48×
		whale	5	32.0	2.2	1.0	1.51×
	Pendulum	verifier-only	3	203.1	7.0	7.0	—
		random	4	57.8	6.8	1.0	2.74×
		whale	4	66.8	8.5	1.0	3.19×
α -CROWN	LinSwitch	verifier-only	5	935.0	69.2	69.2	—
		random	5	91.8	15.4	2.0	10.19×
		whale	5	59.6	10.8	1.0	15.70×
	DoubleWell	<i>(did not complete)</i>					
	Pendulum	verifier-only	4	9559.4	8.0	8.0	—
		random	4	1363.0	6.8	1.0	5.97×
		whale	4	1331.5	8.8	1.0	6.81×

Table 8: End-to-end CEGIS performance per engine, system and counterexample strategy: mean wall-clock time, CEGIS rounds and sound-verifier calls over the n completed seeds. Speed-ups are ratios of seed-matched means against the verifier-only baseline. 39 of the 135 configured arms did not complete (hence the uneven n); in particular the α -CROWN DoubleWell arms are absent, and the IBP Pendulum cells are single-seed and admit no matched comparison. The IBP LinSwitch baseline mean is inflated by a single runaway seed (see text).

Wall time improvements. Stripping out the repeated sound calls changes what a run spends its time on, not merely how long it takes. Averaged over the three engines, sound verification is the largest single cost of a verifier-only run — a mean 62% of its wall time, against 38% for training and loop overhead. A refuter pre-screen inverts this balance: verification falls to a mean 47% of wall time while training-and-overhead rises to 48% and becomes the dominant term, and the refuter’s own

search accounts for only about 5% — a second or two per run at this budget. Once the refuter absorbs the counterexample search, in other words, CEGIS is no longer verification-bound but *training*-bound.

How cleanly this holds depends on the engine’s per-call cost. Under CROWN it is a clean inversion — verification drops from 57% of a baseline run to 46% and retraining overtakes it at 49%. Under IBP the loop is training-bound from the start: IBP’s calls are so cheap that training is already 58% of the verifier-only run and merely edges up to 62% with a refuter, so here the refuter’s payoff is rounds saved rather than call-time skipped. At the far end, α -CROWN’s single sound call is costly enough that verification stays dominant even after the repeats are gone — still 68% of a refuter run on average, and 98% on Pendulum. Verification ceases to dominate exactly when a single sound call is cheap relative to training, which is the regime the pre-screen is built to create.

Difficulty scaling. On Pendulum under CROWN, where a single sound verification costs 22–34 s, the refuter arms synthesise a certificate 2.7–3.2 times faster than the baseline; on LinSwitch under α -CROWN, whose slope optimisation raises the per-call cost to 9–18 s, the speed-up reaches $10.2\times$ for random and $15.7\times$ for whale — the largest in the campaign, because the verifier-only baseline there spirals to 69 rounds (935 s) that the refuter collapses to about 11. Conversely, on the same LinSwitch under plain CROWN a call costs well under a second, so there is nothing worth saving: random search actually *slows the loop down* ($0.85\times$), and whale’s modest $1.16\times$ gain comes from needing fewer rounds rather than from cheaper rounds. DoubleWell sits in between: most seeds certify in one or two rounds, leaving little verification to skip. The extreme case is Pendulum under α -CROWN, where a single sound call costs some 20 *minutes*: the verifier-only baseline makes six to eleven of them — one-and-a-half to three-and-a-half *hours* per run — while each refuter arm makes exactly one and completes in around 22 minutes, a $6.8\times$ speed-up. At this price the pre-screen is the difference between an overnight campaign and a coffee break.

The mechanism is visible in the calls column of Table 8. The baseline pays one

sound verification per round (up to seventy on the α -CROWN LinSwitch), whereas the refuter arms finish with a handful of sound calls at most, and whale with exactly one: the final, unavoidable certification call. The refuters' own cost is negligible at this budget, a second or two per run in total. The saving is therefore the per-call verification cost multiplied by the rounds avoided, which is why the speed-up tracks the engine and system so closely: skipping a dozen sub-second calls buys nothing, while skipping several twenty-minute calls buys hours.

Counterexample leakage. In 12 of 33 completed random runs the refuter declared a candidate clean that the sound verifier then refuted — 18 escaped counterexamples in all, each costing an extra expensive verification round. Whale escaped nothing: in all 34 of its completed runs the first certificate it passed to the verifier was accepted, so every whale run ended with exactly one sound call. Counterexample *quality* matters too, not just detection: on LinSwitch random needed 15.4 rounds on average against the baseline's 11.8 while whale needed 10.8, matching the verifier-quality counterexamples of the baseline.

A reassuring consistency check: whale's round counts are identical, seed for seed, under all three engines (LinSwitch: 11, 8, 9, 10, 16) — because it never lets the sound verifier find a counterexample, its loop trajectory decouples from the verification engine entirely, whose only remaining role is to price the final call. Random's trajectories match across CROWN and α -CROWN (15.4 rounds) but drift under IBP (18.0): each escape hands control back to the engine, and a loose engine returns different, weaker counterexamples.

Engine choice. Per-call costs span more than three orders of magnitude across the campaign: IBP from 0.2 s (LinSwitch) to 11 s (DoubleWell), CROWN from 0.5 s to 34 s, and α -CROWN from 9 s to some 1200 s on Pendulum. Reaching for the cheapest engine instead of a refuter is a false economy, however. IBP's loose enclosures produce weak counterexamples, and on one LinSwitch seed the verifier-only loop spiralled: 179 rounds, 633 s, and a final certificate with a Lipschitz constant of 58 against the 3–7 typical of the other engines. Notably, the pre-screen *rescues* this failure — fed on refuter counterexamples instead of IBP's own, the same seed certifies

in 16 rounds (56 s, $11\times$ faster) under whale. Here the gain is not skipped call-time (IBP’s calls are nearly free) but avoided rounds: the refuter pays off through two distinct channels, skipping expensive sound calls *and* supplying better counterexamples than a loose verifier can produce. A tight engine guarded by a refuter pre-screen obtains tight bounds while paying the expensive call only once.

7.3 Conclusions

Sound verification is the price of a guarantee, but it need not be paid every round. A cheap refuter in front of the verifier reduces CEGIS to a sequence of fast search-and-retrain rounds followed by a single certification call, and the resulting speed-up scales with how expensive that call is: from nothing when verification is sub-second, to $3\text{--}4\times$ when calls cost tens of seconds, to $15.7\times$ on the α -CROWN LinSwitch runaway and a practical necessity on the α -CROWN Pendulum, where 20-minute sound calls turn a multi-hour verifier-only run into a 22-minute one. The benefit flows through two channels: skipping expensive sound calls, and feeding the learner better counterexamples than a loose verifier can produce — the latter rescuing IBP from a 179-round runaway. The quality of the refuter governs how much of this is realised: an uninformed search leaks counterexamples back to the verifier and pads the loop with uninformative rounds, while the tuned whale optimiser matched the verifier’s own counterexample quality at a fraction of its cost, never leaked, and made the loop’s trajectory independent of the engine beneath it. The practical recipe for streamlining CEGIS is therefore to pair the tightest verifier one can afford with a well-tuned refuter pre-screen, rather than to economise on the verifier itself.

7.4 Future Work

The field offers no shortage of hard problems; we highlight the directions that are opened directly by a finding of this thesis.

Nonlinear drift-Lipschitz bounds. Both Lipschitz contributions of Chapter 3 lean on the linearity that keeps the drift piecewise linear, but the principle behind

them — bound the object the verifier consumes, rather than composing bounds on its factors — does not. Running bound propagation over the composed graph $V \circ f - V$ would extend the tight drift constant, and its per-axis refinement, to nonlinear dynamics.

Closing the LinStoch gap. The capability boundary of Chapter 5 leaves exactly one system uncertified by any relaxation-based engine: LinStoch, whose noise-floor equilibrium leaves the relaxations too little margin to work with. Diagnosing that interaction (for instance by decoupling the equilibrium the learner trains against from the one the verifier certifies) is a self-contained problem whose solution would make the bound-propagation family genuinely suite-spanning.

Rescuing sound sampling. The sampling verifiers of Chapter 5 failed on loose inputs rather than broken theory: termination stalled on the magnitude of the Lipschitz and statistical radii. The anisotropic per-axis constants of Chapter 3 plug directly into the bandit’s splitting rule, and variance-adaptive concentration bounds attack the statistical term. Success here would be significant, as it would deliver a sound verifier for genuinely black-box dynamics, which no other family can offer.

Refuter–verifier co-design. The pre-screen of this chapter treats the refuter and the verifier as strangers who share only a training set. A natural next step is to let the refuter’s near-violations warm-start the verifier’s branch-and-bound, seeding splits where the margin is thinnest, and to make the fall-back policy adaptive, spending refuter budget in proportion to how often it has recently let a counterexample escape.

Broader specifications and certificate classes. We claimed in Chapter 2 that our methods extend to safety and reach-avoid specifications without essential modification; that claim deserves an experiment. Likewise, this thesis used piecewise-linear certificates throughout: smooth activations such as $\tanh$ remove the symbolic engines from play but remain within reach of bound propagation, and Lipschitz-constrained architectures interact directly with the margin budget (3.10). Both would re-draw the capability boundary this thesis mapped.

List of Figures

1	Learner–verifier loop for neural certificate synthesis.	16
2	The Lipschitz constant dictates verification cost. <i>Left:</i> each cell must be small enough that the drift cannot cross the margin within it, so cell size $\lesssim \text{margin}/L$; a looser constant forces a finer grid. <i>Right:</i> the box count (3.9) grows as the d -th power of L , so tightening the constant compounds with dimension.	25
3	The budget (3.10) is set by the dynamics: the expected one-step displacement $\mathbb{E}_w\ x' - x\ $ rises from a floor near the equilibrium (which defines our domain-wide, fixed ε).	26
4	MILP verification of a stochastic-linear certificate on the two-dimensional LinStoch system, against the noise discretisation b (bins per noise dimension, so the model carries b^{d_w} noise cells). The <i>phantom</i> regime below $b = 4$ and the steep growth in solve time above it (the cell count itself grows as b^2) are discussed in the text.	39
5	The Gurobi model size (decision variables and binaries) grows as b^{d_w} , since each of the b^{d_w} noise cells must represent and optimise over its own copy of the successor-network encoding. Tightening the over-approximation (larger b) and staying solvable are therefore in direct conflict, and the conflict is exponential in the noise dimension d_w . . .	40

- 6 Branch-and-bound in action: CROWN’s partition of the 2D Pendulum state space at successive depths, for a verified certificate. Each cell is shaded by its sound status — *verified* (drift upper bound ≤ 0 , teal), *undecided* (amber, still to be split), or *violating* (red); the black curve marks the domain boundary and the red dashed circle the goal region. From a single undecided box, `auto_LiRPA` repeatedly halves the cells it cannot yet decide until the whole domain is verified. Tighter relaxations reach this state with far fewer splits than IBP — the compute that the anytime view of Figure 7 prices. 42
- 7 Anytime tightness of the three bound-propagation engines on a verified Pendulum certificate: the sound drift upper bound against wall-clock as branch-and-bound refines. IBP is cheapest per unit compute but loosest, requiring the most domain partitions; α -CROWN reaches the tightest bound at the highest cost; CROWN sits between. All three are sound at every step, as the bound only tightens. 43
- 8 The five BBOB test functions with the global optimum marked in green. Yellow indicates high-quality solutions and purple indicates low-quality solutions. 51
- 9 Normalised regret (log scale) of each refuter at its best hyperparameters, per BBOB function. The multimodal and deceptive functions (Schwefel, Gallagher, Rastrigin) separate the informed methods from the uninformed ones. 53
- 10 Quality versus wall-clock cost at the best hyperparameters (colour = refuter, marker = function; both axes logarithmic). Whale occupies the fast-and-accurate frontier; the Lipschitz methods trade time for accuracy; uninformed search is fast but far worse on the hard landscapes. 54

- 11 Refuting the faux certificates a sound verifier rejected, six refuters compared. *Left*: the fraction of faux certificates each refuter refutes — whale is near-perfect, the other informed and uninformed methods cluster at 91–95%, and DIRECT collapses to 40%. *Right*: per-refuter speed-up over the sound verifier (log): every refuter is one-to-three orders of magnitude cheaper than a sound call, the fast uninformed searches highest. 55
- 12 LinSwitch (Section 4.1.1): the randomly switched linear system. Both switching modes are L_1 -contractive, so every trajectory collapses to the origin irrespective of the switching sequence. 78
- 13 LinStoch (Section 4.1.2): the additive-noise linear system. The Schur-stable drift spirals trajectories inward until the noise floor halts them in a neighbourhood of the origin — the equilibrium ball the certificate must enclose. 78
- 14 DoubleWell (Section 4.1.3): the tilted stochastic gradient flow, shown zoomed onto its true attractor at $\approx (1, 0)$ — the off-origin equilibrium that sets this system apart from the others. 79
- 15 Pendulum (Section 4.1.4): the LQR-controlled inverted pendulum in the $(\theta, \dot{\theta})$ plane. The closed loop is Schur-stable but non-normal, so the forward-invariant domain is the Lyapunov sublevel ellipsoid (black) rather than a box; trajectories spiral into the upright equilibrium. 79
- 16 Evaluation patterns of each refuter over a BBOB landscape at a fixed budget. Uninformed methods (random, grid) spread evaluations evenly; the Lipschitz-aware and population methods (DIRECT, AdaLIPO, whale) concentrate around promising basins, which is what lets them locate deceptive optima the uninformed methods miss. . . . 81

List of Tables

1	The certificate conditions and the guarantees they induce; throughout $V \geq 0$ with $V = 0$ on $\mathcal{X}_G$. This thesis targets the ranking supermartingale, whose condition is the drift inequality (2.13).	13
2	Methods for estimating a network’s Lipschitz constant. <i>Result</i> is the kind of estimate returned: a sound upper bound may safely be used for verification, a lower bound may not; “local” methods bound the constant on a bounded region rather than globally (the relevant notion for verification). <i>Tightness</i> is how close the estimate comes to the true, smallest constant — looseness is not merely aesthetic, since any over-estimate inflates verification cost through the box count (3.9). <i>Cost</i> is the computation needed to produce the estimate, ranging from closed-form per-layer arithmetic to branch-and-bound with worst-case exponential time. LipBaB (bold) is the exact method our experiment builds on.	21
3	Sound drift-Lipschitz estimates (upper bounds on L_D). M1 = product of spectral norms; M2 = LipBaB on V composed via (L_f+1) ; M3 = LipBaB run on the drift $V \circ A - V$. M3 is uniformly tightest.	28
4	Sound anisotropic (per-axis) Lipschitz constants extracted from LipBaB, and the resulting axis-aligned box-count reduction $\prod_i (L_V/L_V(e_i))$. Every $L_V(e_i) \leq L_V$; the saving compounds with dimension.	29
5	The benchmark suite across the three selection axes.	32

6	Best hyperparameters per refuter, by mean normalised regret over the five BBOB functions and three seeds. Lower regret is better; time is the mean wall-clock per search.	52
7	Faux-certificate refutation, six BBOB-tuned refuters versus the sound verifier that rejected each candidate. Columns: the fraction of faux certificates on which a genuine counterexample was found; <i>median</i> wall-clock to a counterexample for the verifier and the refuter, and their ratio; and median counterexample quality (scored drift). Whale is the most reliable and returns the deepest counterexamples; the uninformed searches are the cheapest per call; DIRECT misses most of the linear-system certificates.	55
8	End-to-end CEGIS performance per engine, system and counterexample strategy: mean wall-clock time, CEGIS rounds and sound-verifier calls over the n completed seeds. Speed-ups are ratios of seed-matched means against the verifier-only baseline. 39 of the 135 configured arms did not complete (hence the uneven n); in particular the α -CROWN DoubleWell arms are absent, and the IBP Pendulum cells are single-seed and admit no matched comparison. The IBP LinSwitch baseline mean is inflated by a single runaway seed (see text).	58

Bibliography

- [1] Alessandro Abate, Daniele Ahmed, Alec Edwards, Mirco Giacobbe, and Andrea Peruffo. Fossil: a software tool for the formal synthesis of lyapunov functions and barrier certificates using neural networks. In *Proceedings of the 24th ACM International Conference on Hybrid Systems: Computation and Control (HSCC 2021)*, 2021. URL: <https://dl.acm.org/doi/10.1145/3447928.3456646>.
- [2] Alessandro Abate, Daniele Ahmed, Mirco Giacobbe, and Andrea Peruffo. Formal synthesis of lyapunov neural networks. *IEEE Control Systems Letters*, 5(3):773–778, 2020. URL: <https://arxiv.org/abs/2003.08910>.
- [3] Linda J. S. Allen. An introduction to stochastic epidemic models. In Fred Brauer, Pauline van den Driessche, and Jianhong Wu, editors, *Mathematical Epidemiology*, volume 1945 of *Lecture Notes in Mathematics*, pages 81–130. Springer, 2008. doi:10.1007/978-3-540-78911-6_3.
- [4] Cem Anil, James Lucas, and Roger Grosse. Sorting out lipschitz function approximation. In *Proceedings of the 36th International Conference on Machine Learning (ICML 2019)*, 2019. URL: <https://arxiv.org/abs/1811.05381>.
- [5] Matin Ansari pour, Krishnendu Chatterjee, Thomas A. Henzinger, Mathias Lechner, and Đorđe Žikelić. Learning provably stabilizing neural controllers for discrete-time stochastic systems. In *Automated Technology for Verification and Analysis (ATVA 2023)*, volume 14215 of *Lecture Notes in Computer Science*, pages 357–379. Springer, 2023. URL: <https://arxiv.org/abs/2210.05304>, doi:10.1007/978-3-031-45329-8_17.

- [6] Alexandre Araujo, Aaron Havens, Blaise Delattre, Alexandre Allauzen, and Bin Hu. A unified algebraic perspective on lipschitz neural networks. In *International Conference on Learning Representations (ICLR 2023)*, 2023. URL: <https://arxiv.org/abs/2303.03169>.
- [7] Thom Badings, Wietze Koops, Sebastian Junges, and Nils Jansen. Policy verification in stochastic dynamical systems using logarithmic neural certificates. In *Computer Aided Verification (CAV 2025)*, Lecture Notes in Computer Science. Springer, 2025. URL: <https://arxiv.org/abs/2406.00826>, doi:10.1007/978-3-031-98679-6_16.
- [8] Felix Berkenkamp, Matteo Turchetta, Angela P. Schoellig, and Andreas Krause. Safe model-based reinforcement learning with stability guarantees. In *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, pages 908–918, 2017. URL: <https://arxiv.org/abs/1705.08551>.
- [9] Aritra Bhowmick, Meenakshi D’Souza, and G. Srinivasa Raghavan. Lipbab: Computing exact lipschitz constant of relu networks. In *International Conference on Artificial Neural Networks (ICANN 2021)*, Lecture Notes in Computer Science. Springer, 2021. URL: <https://arxiv.org/abs/2105.05495>.
- [10] Luca Bortolussi, Dimitrios Milios, and Guido Sanguinetti. Smoothed model checking for uncertain continuous time markov chains. *Information and Computation*, 2016. Preprint arXiv:1402.1450 (2014). URL: <https://arxiv.org/abs/1402.1450>.
- [11] Debasish Chatterjee and Soumik Pal. An excursion-theoretic approach to stability of discrete-time stochastic hybrid systems. *Applied Mathematics & Optimization*, 63(2):217–237, 2011. Preprint arXiv:0901.2269, submitted January 2009. doi:10.1007/s00245-010-9117-6.
- [12] Krishnendu Chatterjee, Thomas A. Henzinger, Mathias Lechner, and Đorđe Žikelić. A learner-verifier framework for neural network controllers and certificates of stochastic systems. In *Tools and Algorithms for the Construction and*

- Analysis of Systems (TACAS 2023)*, volume 13993 of *Lecture Notes in Computer Science*, pages 3–25. Springer, 2023. URL: https://link.springer.com/chapter/10.1007/978-3-031-30823-9_1.
- [13] Leonardo de Moura and Nikolaj Bjørner. Z3: An efficient SMT solver. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2008)*, volume 4963 of *Lecture Notes in Computer Science*, pages 337–340. Springer, 2008. doi:10.1007/978-3-540-78800-3_24.
- [14] DeepMind. `jax_verify`: Neural network verification in `jax`. https://github.com/deepmind/jax_verify, 2020. Accessed: 2026-07-10.
- [15] Alec Edwards, Andrea Peruffo, and Alessandro Abate. Fossil 2.0: Formal certificate synthesis for the verification and control of dynamical models. In *Proceedings of the 27th ACM International Conference on Hybrid Systems: Computation and Control (HSCC 2024)*, 2024. URL: <https://arxiv.org/abs/2311.09793>.
- [16] Mahyar Fazlyab, Alexander Robey, Hamed Hassani, Manfred Morari, and George J. Pappas. Efficient and accurate estimation of lipschitz constants for deep neural networks. In *Advances in Neural Information Processing Systems 32 (NeurIPS 2019)*, 2019. URL: <https://arxiv.org/abs/1906.04893>.
- [17] Sicun Gao, Jeremy Avigad, and Edmund M. Clarke. δ -complete decision procedures for satisfiability over the reals. In *Automated Reasoning — 6th International Joint Conference (IJCAR 2012)*, volume 7364 of *Lecture Notes in Computer Science*, pages 286–300. Springer, 2012. URL: <https://arxiv.org/abs/1204.3513>.
- [18] Sicun Gao, Soonho Kong, and Edmund M. Clarke. dReal: An SMT solver for nonlinear theories over the reals. In *24th International Conference on Automated Deduction (CADE-24)*, volume 7898 of *Lecture Notes in Computer Science*, pages 208–214. Springer, 2013. doi:10.1007/978-3-642-38574-2_14.

- [19] Henry Gouk, Eibe Frank, Bernhard Pfahringer, and Michael J. Cree. Regularisation of neural networks by enforcing Lipschitz continuity. *Machine Learning*, 110(2):393–416, 2021. arXiv:1804.04368. doi:10.1007/s10994-020-05929-w.
- [20] Sven Gowal, Krishnamurthy Dvijotham, Robert Stanforth, Rudy Bunel, Chongli Qin, Jonathan Uesato, Relja Arandjelovic, Timothy Mann, and Pushmeet Kohli. On the effectiveness of interval bound propagation for training verifiably robust models. In *arXiv preprint arXiv:1810.12715*, 2018. URL: <https://arxiv.org/abs/1810.12715>.
- [21] Ishaan Gulrajani, Faruk Ahmed, Martin Arjovsky, Vincent Dumoulin, and Aaron Courville. Improved training of wasserstein gans. In *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, 2017. URL: <https://arxiv.org/abs/1704.00028>.
- [22] Gurobi Optimization, LLC. Gurobi optimizer reference manual, 2025. Version 13.0. URL: <https://www.gurobi.com>.
- [23] N. Hansen, A. Auger, R. Ros, O. Mersmann, T. Tušar, and D. Brockhoff. COCO: A platform for comparing continuous optimizers in a black-box setting. *Optimization Methods and Software*, 36:114–144, 2021. doi:10.1080/10556788.2020.1808977.
- [24] Yujia Huang, Huan Zhang, Yuanyuan Shi, J. Zico Kolter, and Anima Anandkumar. Training certifiably robust neural networks with efficient local lipschitz bounds. In *Advances in Neural Information Processing Systems 34 (NeurIPS 2021)*, 2021. URL: <https://arxiv.org/abs/2111.01395>.
- [25] Donald R. Jones, Cary D. Perttunen, and Bruce E. Stuckman. Lipschitzian optimization without the lipschitz constant. *Journal of Optimization Theory and Applications*, 79(1):157–181, 1993. doi:10.1007/BF00941892.
- [26] Matt Jordan and Alexandros G. Dimakis. Exactly computing the local lipschitz constant of relu networks. In *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, 2020. URL: <https://arxiv.org/abs/2003.01219>.

- [27] Philippe Jorion. *Value at Risk: The New Benchmark for Managing Financial Risk*. McGraw-Hill, New York, 3rd edition, 2007.
- [28] Dejan Jovanović and Leonardo de Moura. Solving non-linear arithmetic. In *6th International Joint Conference on Automated Reasoning (IJCAR 2012)*, volume 7364 of *Lecture Notes in Computer Science*, pages 339–354. Springer, 2012. doi:10.1007/978-3-642-31365-3_27.
- [29] Guy Katz, Clark Barrett, David L. Dill, Kyle Julian, and Mykel J. Kochenderfer. Reluplex: An efficient smt solver for verifying deep neural networks. In *29th International Conference on Computer Aided Verification (CAV 2017)*, volume 10426 of *Lecture Notes in Computer Science*, pages 97–117, 2017. URL: <https://arxiv.org/abs/1702.01135>.
- [30] Ker-I Ko. *Complexity Theory of Real Functions*. Progress in Theoretical Computer Science. Birkhäuser Boston, 1991. URL: https://openlibrary.org/books/OL27025512M/Complexity_Theory_of_Real_Functions.
- [31] H. A. Kramers. Brownian motion in a field of force and the diffusion model of chemical reactions. *Physica*, 7(4):284–304, 1940. doi:10.1016/S0031-8914(40)90098-2.
- [32] Simon Kuang, Yuezhu Xu, S. Sivaranjani, and Xinfan Lin. Demystifying lipschitz verification: positive matrices, negative results. *arXiv preprint arXiv:2603.28113*, 2026. URL: <https://arxiv.org/abs/2603.28113>.
- [33] Mathias Lechner, Đorđe Žikelić, Krishnendu Chatterjee, and Thomas A. Henzinger. Stability verification in stochastic control systems via neural network supermartingales. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI 2022)*, volume 36, pages 7326–7336, 2022. URL: <https://arxiv.org/abs/2112.09495>.
- [34] Aleksandr Mikhailovich Lyapunov. The general problem of the stability of motion. *International Journal of Control*, 55(3):531–534, 1992. English translation of the 1892 original. doi:10.1080/00207179208934253.

- [35] Cédric Malherbe and Nicolas Vayatis. Global optimization of Lipschitz functions. In *Proceedings of the 34th International Conference on Machine Learning (ICML 2017)*, volume 70 of *Proceedings of Machine Learning Research*, pages 2314–2323. PMLR, 2017. URL: <https://arxiv.org/abs/1703.02628>.
- [36] Ali Mesbah. Stochastic model predictive control: An overview and perspectives for future research. *IEEE Control Systems Magazine*, 36(6):30–44, 2016. doi: 10.1109/MCS.2016.2602087.
- [37] Seyedali Mirjalili and Andrew Lewis. The whale optimization algorithm. *Advances in Engineering Software*, 95:51–67, 2016. URL: <https://www.sciencedirect.com/science/article/pii/S0965997816300163>, doi: 10.1016/j.advengsoft.2016.01.008.
- [38] Takeru Miyato, Toshiki Kataoka, Masanori Koyama, and Yuichi Yoshida. Spectral normalization for generative adversarial networks. In *International Conference on Learning Representations (ICLR 2018)*, 2018. URL: <https://arxiv.org/abs/1802.05957>.
- [39] Pablo A. Parrilo. *Structured Semidefinite Programs and Semialgebraic Geometry Methods in Robustness and Optimization*. PhD thesis, California Institute of Technology, 2000.
- [40] Patricia Pauli, Dennis Gramlich, and Frank Allgöwer. Lipschitz constant estimation for general neural network architectures using control tools. *arXiv preprint arXiv:2405.01125*, 2024. URL: <https://arxiv.org/abs/2405.01125>.
- [41] Andrea Peruffo, Daniele Ahmed, and Alessandro Abate. Automated and formal synthesis of neural barrier certificates for dynamical models. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2021)*, volume 12651 of *Lecture Notes in Computer Science*. Springer, 2021. URL: <https://arxiv.org/abs/2007.03251>.
- [42] Bernd Prach and Christoph H. Lampert. Almost-orthogonal layers for efficient general-purpose Lipschitz networks. In *Computer Vision — ECCV 2022*, vol-

- ume 13681 of *Lecture Notes in Computer Science*, pages 350–365. Springer, 2022. arXiv:2208.03160. doi:10.1007/978-3-031-19803-8_21.
- [43] Luca Schenato, Bruno Sinopoli, Massimo Franceschetti, Kameshwar Poolla, and S. Shankar Sastry. Foundations of control and estimation over lossy networks. *Proceedings of the IEEE*, 95(1):163–187, 2007. doi:10.1109/JPROC.2006.887306.
- [44] Zhouxing Shi, Yihan Wang, Huan Zhang, Zico Kolter, and Cho-Jui Hsieh. Efficiently computing local lipschitz constants of neural networks via bound propagation. In *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*, 2022. URL: <https://arxiv.org/abs/2210.07394>.
- [45] Bruno O. Shubert. A sequential method seeking the global maximum of a function. *SIAM Journal on Numerical Analysis*, 9:379–388, 1972. doi:10.1137/0709036.
- [46] Asher Trockman and J. Zico Kolter. Orthogonalizing convolutional layers with the cayley transform. In *International Conference on Learning Representations (ICLR 2021)*, 2021. URL: <https://arxiv.org/abs/2104.07167>.
- [47] Yusuke Tsuzuku, Issei Sato, and Masashi Sugiyama. Lipschitz-margin training: Scalable certification of perturbation invariance for deep neural networks. In *Advances in Neural Information Processing Systems 31 (NeurIPS 2018)*, 2018. URL: <https://arxiv.org/abs/1802.04034>.
- [48] Aladin Virmaux and Kevin Scaman. Lipschitz regularity of deep neural networks: analysis and efficient estimation. In *Advances in Neural Information Processing Systems 31 (NeurIPS 2018)*, 2018. URL: <https://arxiv.org/abs/1805.10965>.
- [49] Đorđe Žikelić, Mathias Lechner, Thomas A. Henzinger, and Krishnendu Chatterjee. Learning control policies for stochastic systems with reach-avoid guarantees. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI*

- 2023), volume 37, pages 11926–11935, 2023. URL: <https://arxiv.org/abs/2210.05308>.
- [50] Shiqi Wang, Huan Zhang, Kaidi Xu, Xue Lin, Suman Jana, Cho-Jui Hsieh, and J. Zico Kolter. Beta-crown: Efficient bound propagation with per-neuron split constraints for complete and incomplete neural network robustness verification. In *Advances in Neural Information Processing Systems 34 (NeurIPS 2021)*, 2021. URL: <https://arxiv.org/abs/2103.06624>.
- [51] David Williams. *Probability with Martingales*. Cambridge University Press, 1991. doi:10.1017/CB09780511813658.
- [52] Min Wu, Matthew Wicker, Wenjie Ruan, Xiaowei Huang, and Marta Kwiatkowska. A game-based approximate verification of deep neural networks with provable guarantees. *Theoretical Computer Science*, 807:298–329, 2020. URL: <https://arxiv.org/abs/1807.03571>.
- [53] Taoran Wu, Dominik Wagner, Jingduo Pan, Luke Ong, Arvind Easwaran, and Bai Xue. Pac one-step safety certification for black-box discrete-time stochastic systems. *arXiv preprint arXiv:2512.05549*, 2025. URL: <https://arxiv.org/abs/2512.05549>.
- [54] Kaidi Xu, Zhouxing Shi, Huan Zhang, Yihan Wang, Kai-Wei Chang, Minlie Huang, Bhavya Kailkhura, Xue Lin, and Cho-Jui Hsieh. Automatic perturbation analysis for scalable certified robustness and beyond. In *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, 2020. URL: <https://arxiv.org/abs/2002.12920>.
- [55] Kaidi Xu, Huan Zhang, Shiqi Wang, Yihan Wang, Suman Jana, Xue Lin, and Cho-Jui Hsieh. Fast and complete: Enabling complete neural network verification with rapid and massively parallel incomplete verifiers. In *International Conference on Learning Representations (ICLR 2021)*, 2021. URL: <https://arxiv.org/abs/2011.13824>.

- [56] Yuezhu Xu and S. Sivaranjani. Eclipse: Efficient compositional lipschitz constant estimation for deep neural networks. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, 2024. URL: <https://arxiv.org/abs/2404.04375>.
- [57] Huan Zhang, Tsui-Wei Weng, Pin-Yu Chen, Cho-Jui Hsieh, and Luca Daniel. Efficient neural network robustness certification with general activation functions. In *Advances in Neural Information Processing Systems 31 (NeurIPS 2018)*, 2018. URL: <https://arxiv.org/abs/1811.00866>.
- [58] Huan Zhang, Pengchuan Zhang, and Cho-Jui Hsieh. Recurjac: An efficient recursive algorithm for bounding jacobian matrix of neural networks and its applications. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI 2019)*, 2019. URL: <https://arxiv.org/abs/1810.11783>.

Appendix A

2D Benchmark Phase Portraits

Phase portraits of the four 2D benchmark systems introduced in Chapter 4. In each panel the faint streamlines trace the *expected* one-step drift $\mathbb{E}_w[f(x, w)] - x$ — estimated by Monte-Carlo, so they are valid for additive, switched and state-dependent noise alike — while the coloured curves are stochastic sample trajectories rolled out from random starts inside the domain (dark to light with elapsed time). The domain boundary is drawn in black and the equilibrium region as a red dashed circle.

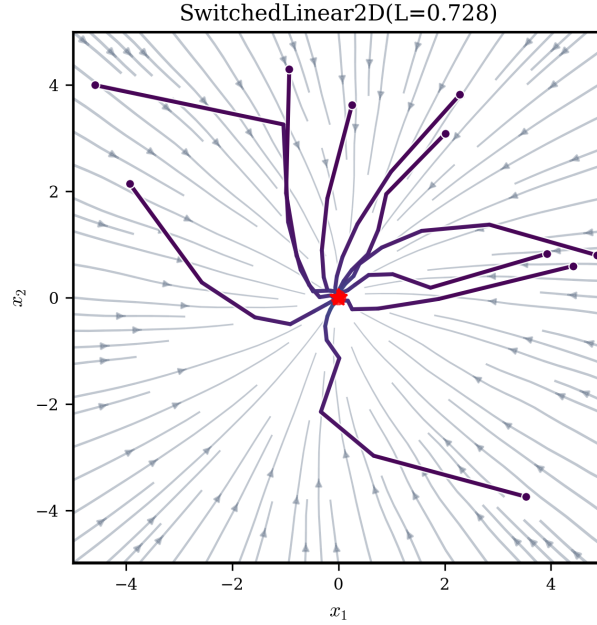


Figure 12: LinSwitch (Section 4.1.1): the randomly switched linear system. Both switching modes are L_1 -contractive, so every trajectory collapses to the origin irrespective of the switching sequence.

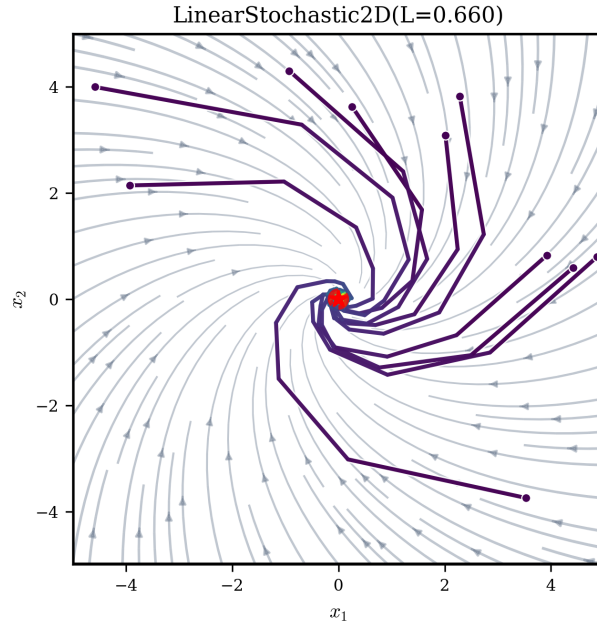


Figure 13: LinStoch (Section 4.1.2): the additive-noise linear system. The Schur-stable drift spirals trajectories inward until the noise floor halts them in a neighbourhood of the origin — the equilibrium ball the certificate must enclose.

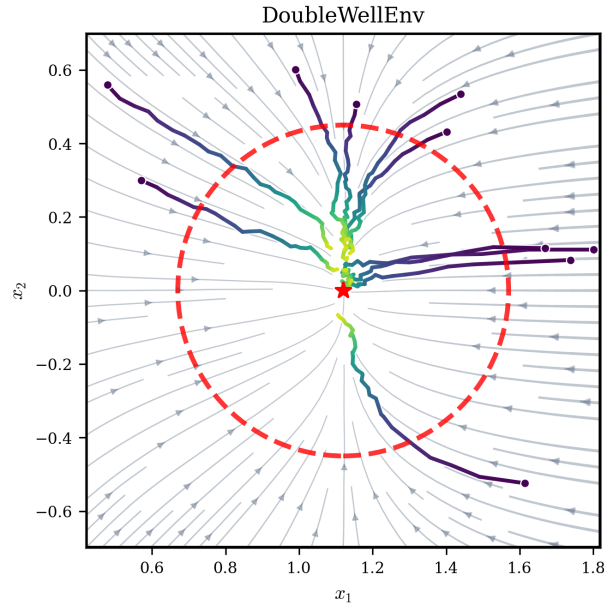


Figure 14: DoubleWell (Section 4.1.3): the tilted stochastic gradient flow, shown zoomed onto its true attractor at $\approx (1, 0)$ — the off-origin equilibrium that sets this system apart from the others.

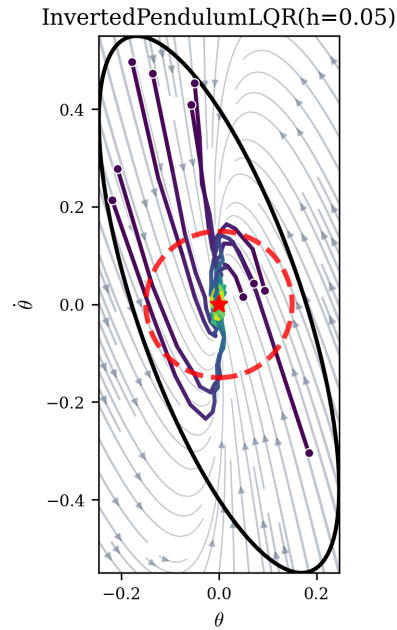


Figure 15: Pendulum (Section 4.1.4): the LQR-controlled inverted pendulum in the $(\theta, \dot{\theta})$ plane. The closed loop is Schur-stable but non-normal, so the forward-invariant domain is the Lyapunov sublevel ellipsoid (black) rather than a box; trajectories spiral into the upright equilibrium.

Appendix B

Examples of Refuter Search Patterns

This appendix supplements the refuter benchmark of Section 6.2.1 with a qualitative view of how each method spends its evaluation budget. Figure 16 overlays the sampled points of every refuter on the five BBOB landscapes, at the same fixed budget, making visible the difference between uninformed search and the informed methods that concentrate around promising basins.

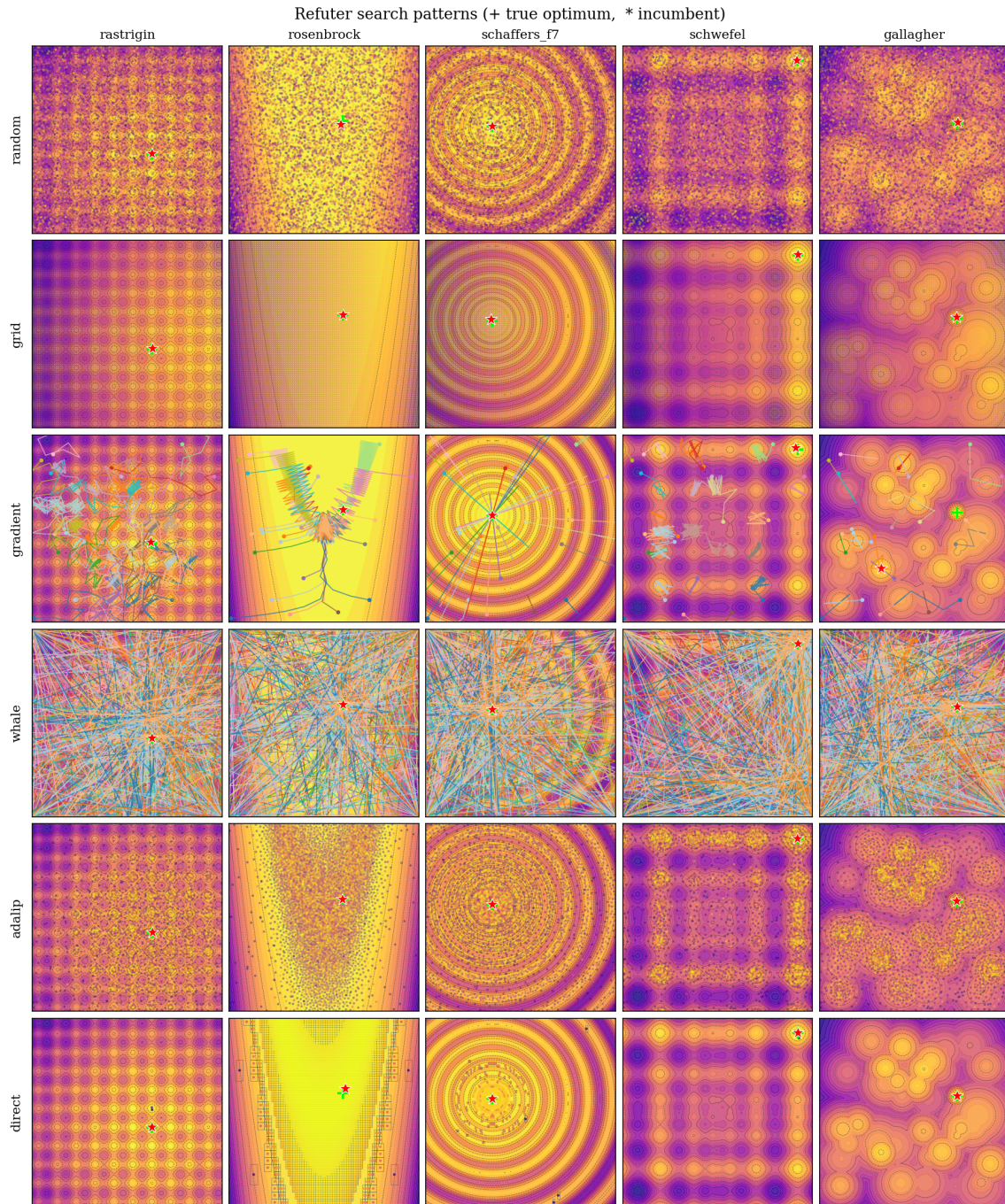


Figure 16: Evaluation patterns of each refuter over a BBOB landscape at a fixed budget. Uninformed methods (random, grid) spread evaluations evenly; the Lipschitz-aware and population methods (DIRECT, AdaLIPO, whale) concentrate around promising basins, which is what lets them locate deceptive optima the uninformed methods miss.